Name: ENG Piseth

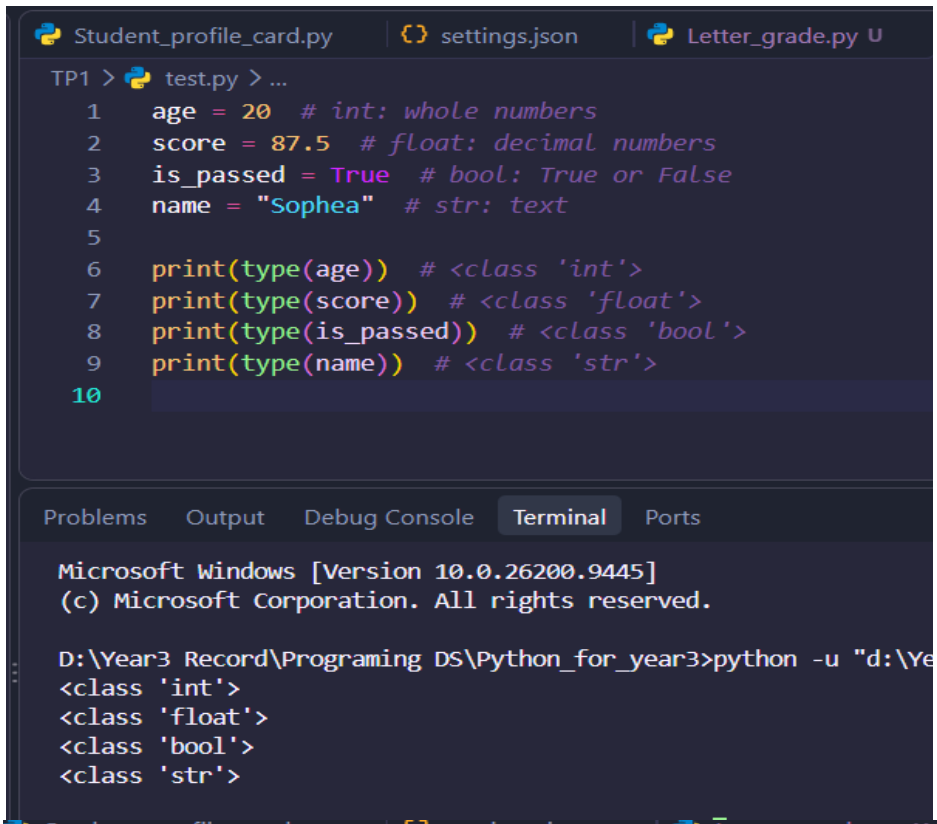
ID: e20240972

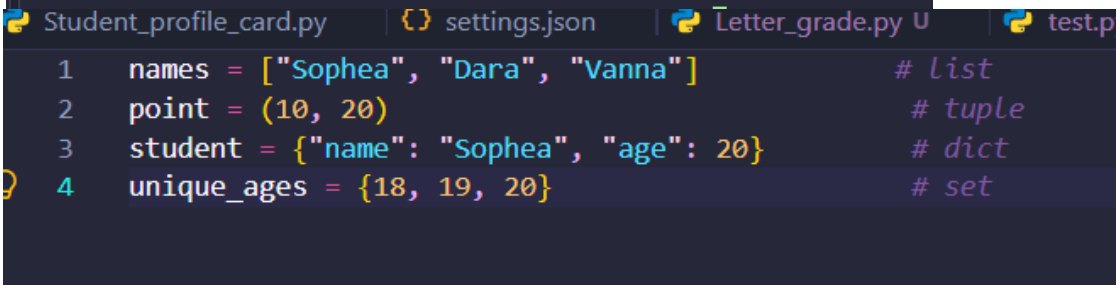
Lecturer: VANN VISAL

Group 1: Python Basic Data Types

a. Primitive/Built-in Data Types (int, float, bool, str)

- These are the basic building blocks of data in Python.



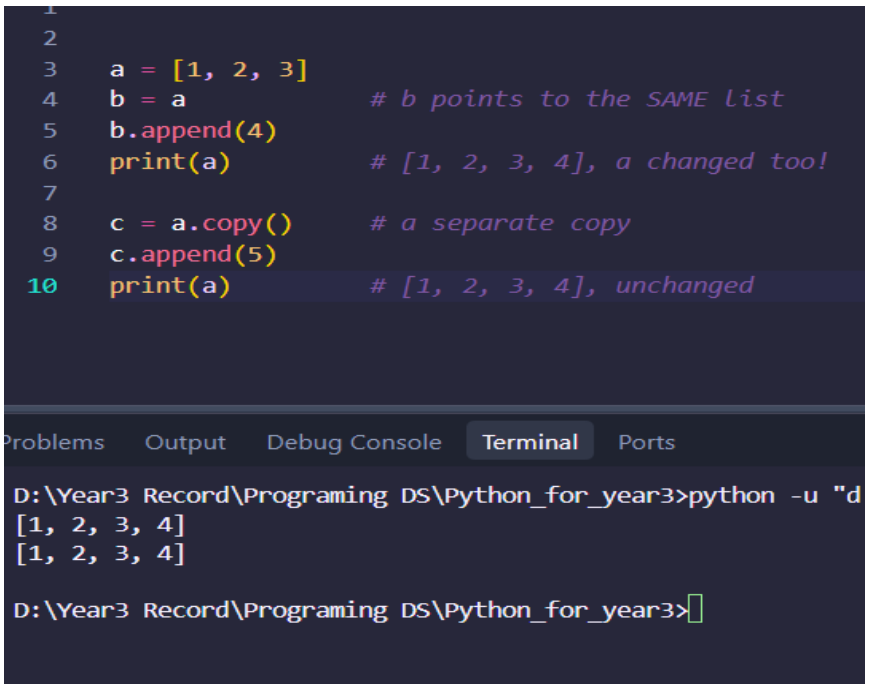


b. Collection and Reference Types (list, tuple, dict, set, objects)

These hold multiple values.

Type	*Ordered?*	*Mutable?*	*Duplicates?*	*Notes*
List	Yes	Yes	Yes	Indexed with []
Tuple	Yes	No	Yes	Fixed once created
Dict	Yes (insertion order)	Yes	Unique keys	Key-value pairs
Set	No	Yes	No	Fast membership tests

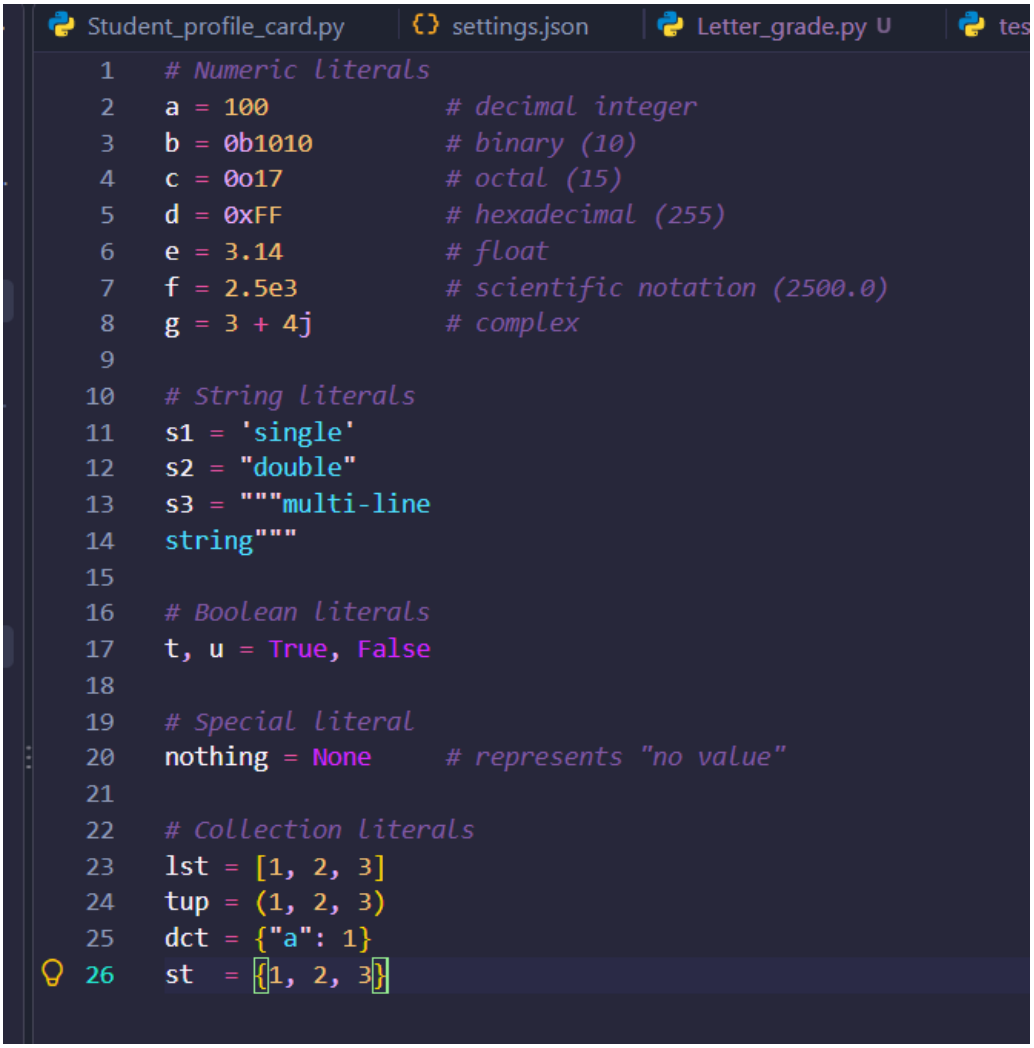
- **Objects and references:** In Python, everything is an object. Variables hold references to objects rather than the data itself.
- Mutable types (list, dict, set) can change in place. Immutable types (int,



float, bool, str, tuple) cannot.

c. Literals in Python (numeric, string, Boolean, None, collection literals)
Practical example: storing student names, ages, scores, and student records.

- A literal is a fixed value written directly in the code.



Example of student records:

Student_profile_card.py | settings.json | Letter_grade.py U | test.py U

```python
# Individual values (primitive types)
student_name = "Sophea"
student_age = 20
student_score = 88.5
has_passed = student_score >= 50

# Lists of names, ages, and scores
names  = ["Sophea", "Dara", "Vanna"]
ages   = [20, 21, 19]
scores = [88.5, 72.0, 45.5]

# A single student record (dict)
student = {
    "name": "Sophea",
    "age": 20,
    "score": 88.5,
    "passed": True,
    "address": None          # unknown yet
}

# A list of student records (collection of dicts)
students = [
    {"name": "Sophea", "age": 20, "score": 88.5},
    {"name": "Dara",   "age": 21, "score": 72.0},
    {"name": "Vanna",  "age": 19, "score": 45.5},
]

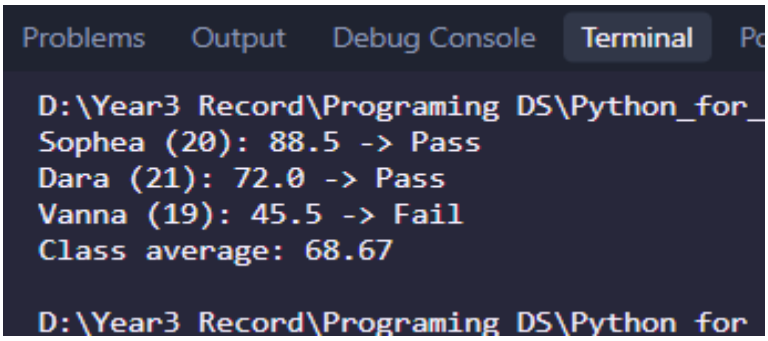
# A set to find the unique ages
unique_ages = set(ages)

# A tuple for fixed data (name, ID)
record_id = ("Sophea", 1001)

# Using the data
for s in students:
    status = "Pass" if s["score"] >= 50 else "Fail"
    print(f"{s['name']} ({s['age']}): {s['score']} -> {status}")

average = sum(s["score"] for s in students) / len(students)
print(f"Class average: {average:.2f}")
```

Output:

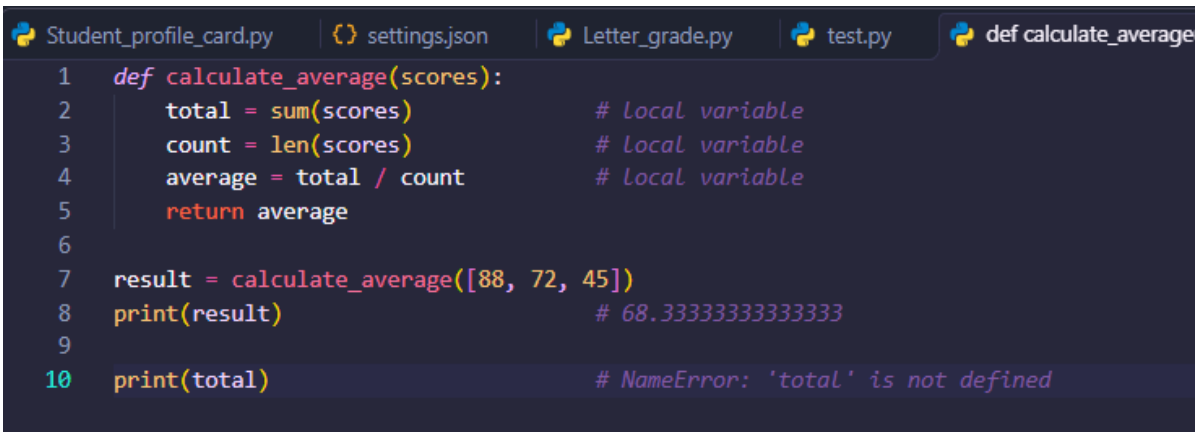


Key points:

- Use primitives for single values, and collections to group them.
- Choose list for ordered, changeable data, tuple for fixed data, dict for labeled data, and set for unique values.
- Remember that mutable objects share references when assigned

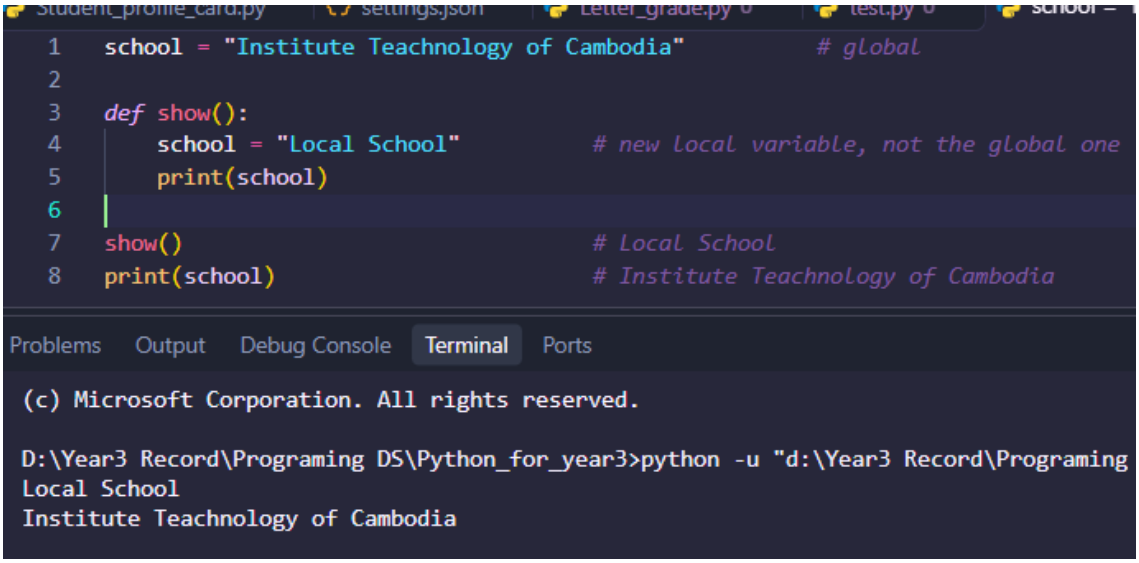
Group 2: Python Variables and Attributes
a. Local Variables

- A local variable is created inside a function and exists only while that function runs. It cannot be accessed from outside.



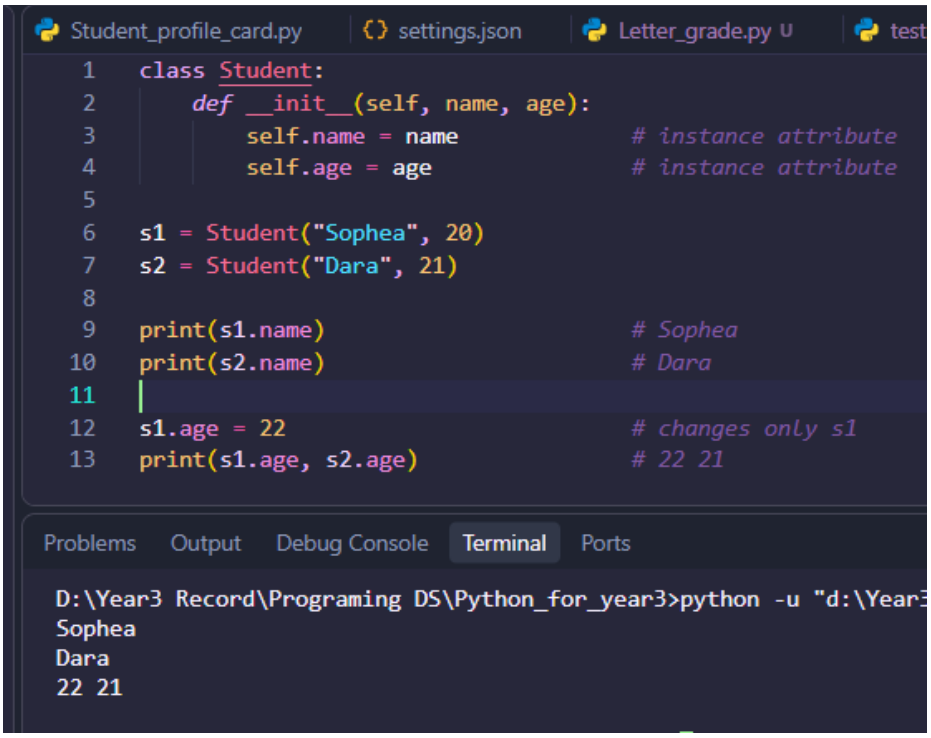
Key points:

- Each function call gets its own fresh set of local variables.
- Parameters (like scores) are also local variables.
- A local variable can shadow a global one with the same name, without changing it.



b. Instance Attributes

- An instance attribute belongs to one specific object. It is usually created in __init__ using self, so every object stores its own values.

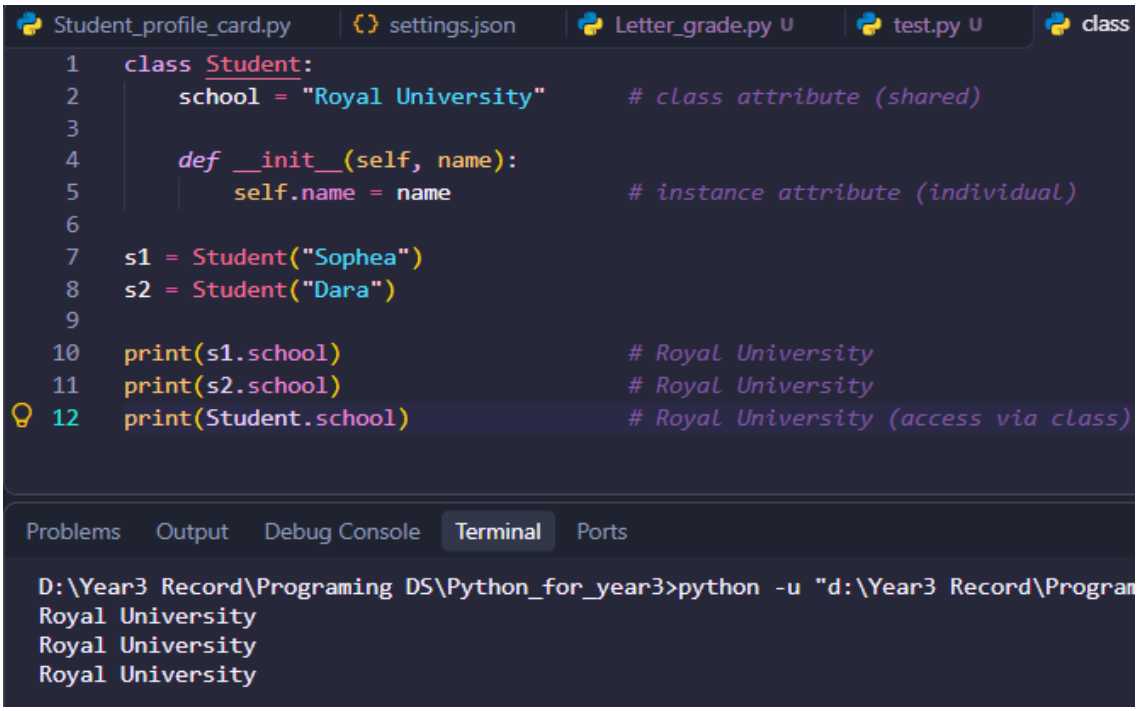


Key points:

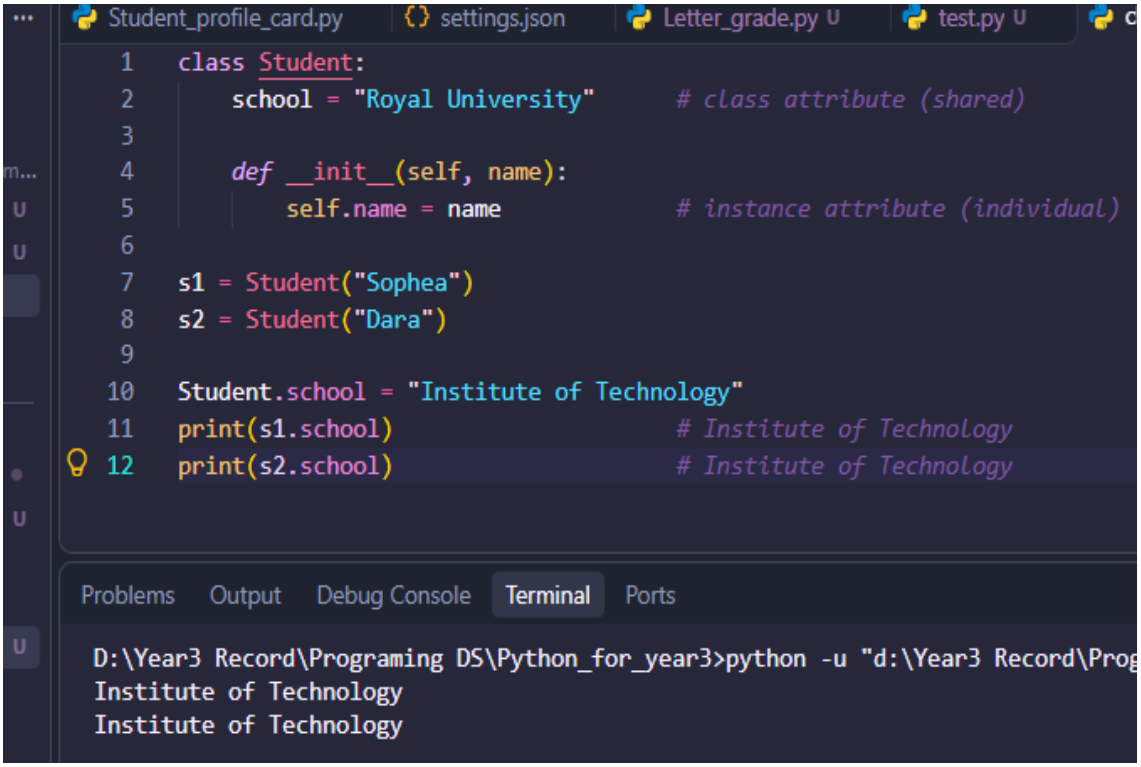
- self refers to the specific object being created or used.
- Changing one object's attribute does not affect other objects.
- Each instance stores its attributes in its own __dict__.

c. Class Attributes (shared by all instances)

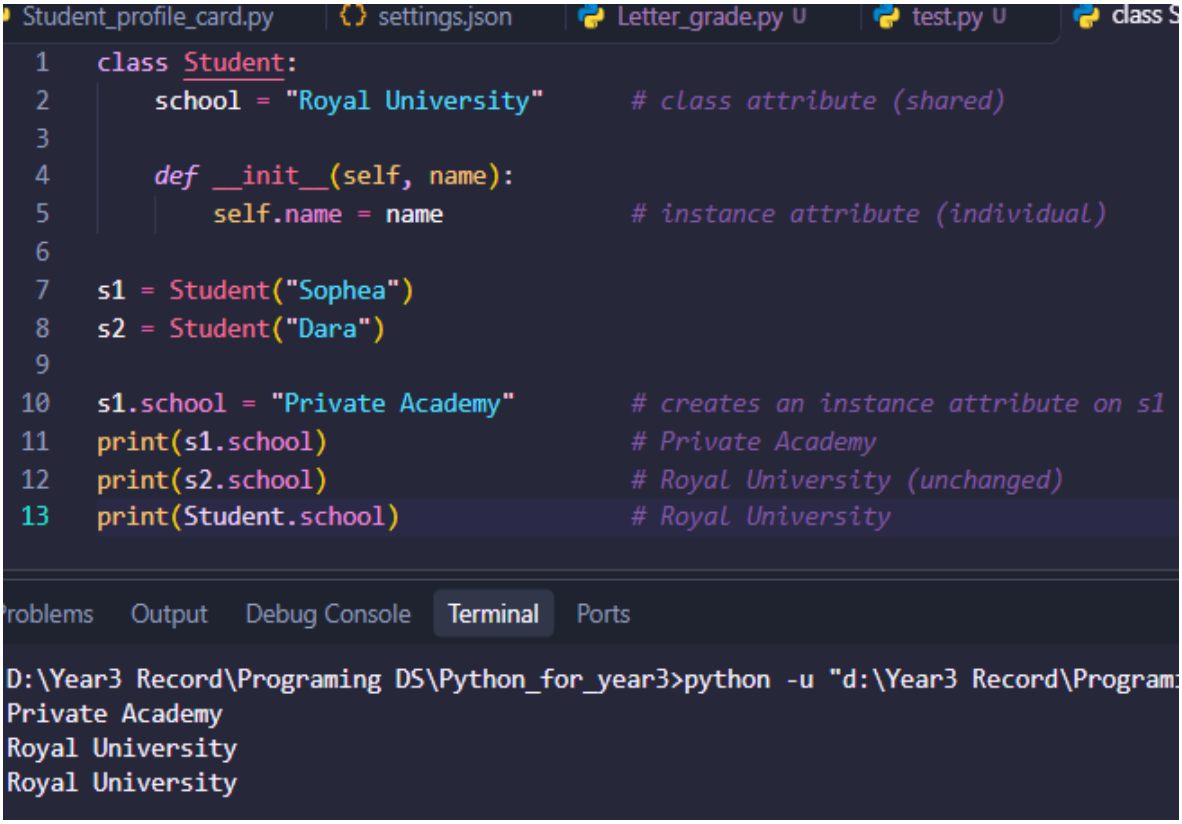
- A class attribute is defined directly in the class body, outside any method. All instances share the same value.



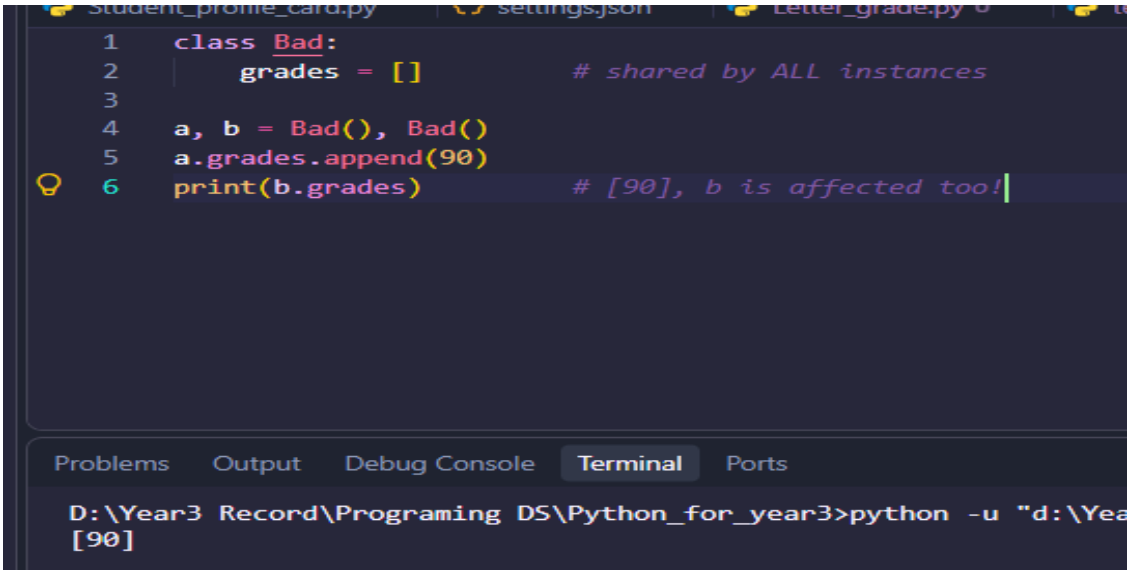
Changing a shared value: Change it through the class, and every instance sees the update.



Common pitfall: Assigning through an instance creates a new instance attribute that hides the class attribute for that object only.



Another pitfall: Avoid mutable class attributes (like lists) unless you want them shared.



Comparison:

Feature	*Local Variable*	*Instance Attribute*	*Class Attribute*
Defined in	Function body	__init__ / methods via self	Class body
Belongs to	One function call	One object	The class (shared)
Lifetime	Until function ends	Until object is deleted	Until class is deleted
Access	Inside function only	object.attr	Class.attr or object.attr
Unique per object?	N/A	Yes	No, shared

- Practical Example: Student Class

```python
class Student:
    school = "Royal University"          # class attribute: shared by all students
    student_count = 0                    # class attribute: counts all students

    def __init__(self, name, age, scores):
        self.name = name                 # instance attributes: unique to each student
        self.age = age
        self.scores = scores
        Student.student_count += 1       # update the shared counter

    def average(self):
        total = sum(self.scores)         # local variable
        return total / len(self.scores)

    def introduce(self):
        avg = self.average()             # local variable
        print(f"{self.name}, age {self.age}, studies at {Student.school}. "
              f"Average: {avg:.2f}")
```

```python
# Create students
s1 = Student("Sophea", 20, [88, 92, 85])
s2 = Student("Dara", 21, [72, 65, 80])
s3 = Student("Vanna", 19, [45, 60, 55])

s1.introduce()
s2.introduce()
s3.introduce()

print("Total students:", Student.student_count)

# Change the school for everyone
Student.school = "Institute of Technology"
s2.introduce()
```

- Output:

```
Microsoft Windows [Version 10.0.26200.9445]
(c) Microsoft Corporation. All rights reserved.

D:\Year3 Record\Programing DS\Python_for_year3>python -u "d:\Year3 Record\Programing DS\Python_for_year3
Sophea, age 20, studies at Royal University. Average: 88.33
Dara, age 21, studies at Royal University. Average: 72.33
Vanna, age 19, studies at Royal University. Average: 53.33
Total students: 3
Dara, age 21, studies at Institute of Technology. Average: 72.33
```

Key Takeaways

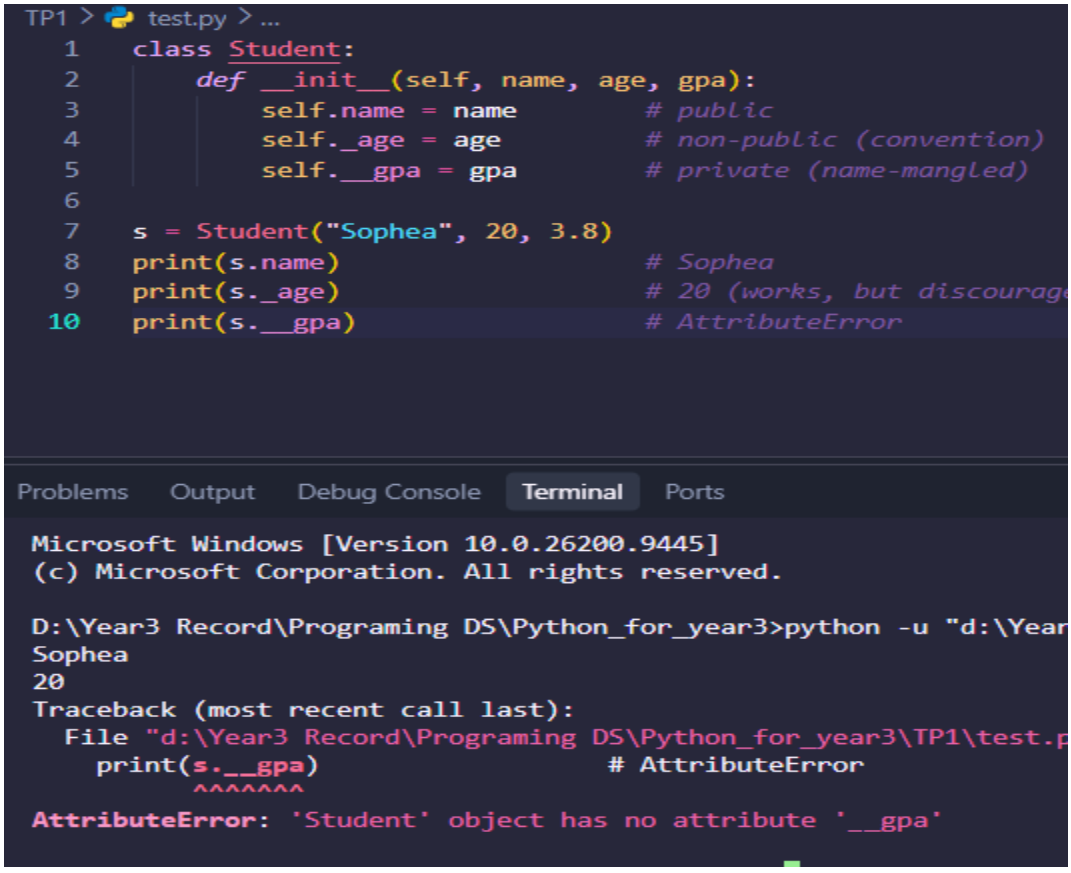
- **Local variables** are temporary and private to a function call.
- **Instance attributes** (self.x) store data unique to each object.
- **Class attributes** store data shared by all objects, and are best changed through the class name (Student.school = ...).
- Python looks up an attribute on the instance first, then on the class.

Group 3: Python Modifiers and Access Control — Part 1

a. Public, Non-Public, and Private Naming Conventions

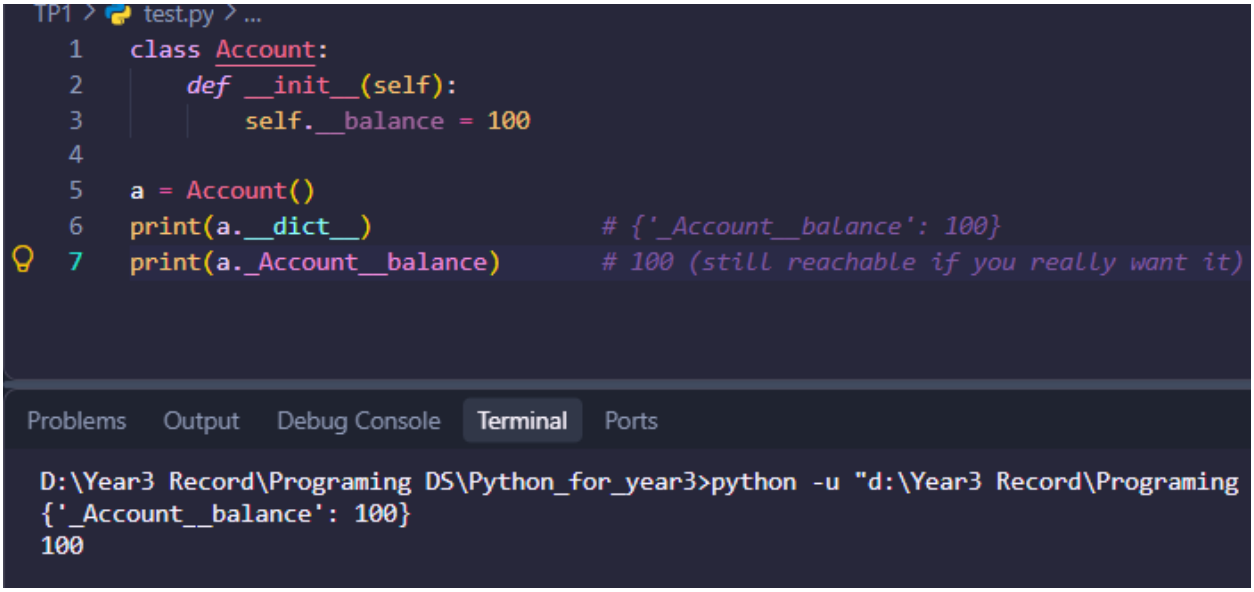
- Python has no public or private keywords. Instead, the **name itself** signals how an attribute or method should be used.

Style	***Example***	***Meaning***
Public	name	Part of the official interface; free to use anywhere
Non-public (protected)	_name	Internal use; "please don't touch from outside"
Private	__name	Name-mangled to reduce accidental access and clashes
Special (dunder)	__init__	Python's own hooks; not private, not mangled



b. Access Control by Convention and Name Mangling

- **By convention:** A single underscore is a message to other programmers. Python does not enforce it.
- **By name mangling:** Inside a class, __balance is rewritten to _ClassName__balance. This stops accidental access and prevents subclasses from accidentally overriding the attribute.



Key points:

- Mangling is **not security**. It is protection against mistakes, not against determined users.
- Names that **end** with two underscores (like __init__) are not mangled.
- Python's philosophy: "We are all consenting adults here." Offer a clear public interface and trust users to respect it.

c. The @staticmethod Decorator

- A static method belongs to the class's namespace but receives **neither self nor cls**. It behaves like a regular function that is grouped inside the class because it is logically related.

```python
class MathTools:
    @staticmethod
    def add(a, b):
        return a + b

print(MathTools.add(3, 4))       # 7 (no object needed)
print(MathTools().add(3, 4))     # 7 (also works via an instance)
```

```
D:\Year3 Record\Programing DS\Python_for_year3>python -u "d:\Year3 Record\P
7
7
```

- **Use it when** the method does not need any object or class data, such as validation or utility helpers.

d. The @classmethod Decorator

- A class method receives the **class itself** as its first argument (cls). It can read or modify class attributes and is commonly used for **alternative constructors**.

```python
class Student:
    school = "Royal University"

    def __init__(self, name):
        self.name = name

    @classmethod
    def from_string(cls, text):
        name = text.strip().title()
        return cls(name)                # creates an instance of the class

    @classmethod
    def change_school(cls, new_name):
        cls.school = new_name

s = Student.from_string("  dara ")
print(s.name)                           # Dara
Student.change_school("Institute of Technology")
print(s.school)                         # Institute of Technology
```

```
D:\Year3 Record\Programing DS\Python_for_year3>python -u "d:\Year3 Record\Programin
Dara
Institute of Technology
```

- **Why cls instead of the class name?** If a subclass calls the method, cls is the subclass, so inheritance works correctly.

Comparison of Method Types:

Feature	*Instance Method*	*@classmethod*	*@staticmethod*
First parameter	self	cls	none
Access instance data	Yes	No	No
Access class data	Yes (via self/class)	Yes	Only by naming the class
Typical use	Behavior of one object	Alternative constructors, class-wide changes	Utility/helper functions

> Practical example: controlling access to a BankAccount class and defining class-level methods.

```
P1 > test.py > ...
1   class BankAccount:
2       bank_name = "National Bank"          # public class attribute
3       _total_accounts = 0                  # non-public class attribute
4
5       def __init__(self, owner, balance=0):
6           self.owner = owner               # public
7           self._account_type = "Savings"   # non-public
8           self.__balance = balance         # private
9           BankAccount._total_accounts += 1
10
11      # ---- Public interface (controlled access to the private balance) ----
12      def deposit(self, amount):
13          if not BankAccount.is_valid_amount(amount):
14              raise ValueError("Amount must be a positive number")
15          self.__balance += amount
16
17      def withdraw(self, amount):
18          if not BankAccount.is_valid_amount(amount):
19              raise ValueError("Amount must be a positive number")
```

```
    def withdraw(self, amount):
        if not BankAccount.is_valid_amount(amount):
            raise ValueError("Amount must be a positive number")
        if amount > self.__balance:
            raise ValueError("Insufficient funds")
        self.__balance -= amount

    def get_balance(self):
        return self.__balance

    # ---- Static method: pure validation helper ----
    @staticmethod
    def is_valid_amount(amount):
        return isinstance(amount, (int, float)) and amount > 0

    # ---- Class methods ----
    @classmethod
    def from_string(cls, data):
        owner, balance = data.split(",")
        return cls(owner.strip(), float(balance))

    @classmethod
    def get_total_accounts(cls):
        return cls._total_accounts

```

```
# Using the class
acc1 = BankAccount("Sophea", 500)
acc2 = BankAccount.from_string("Dara, 1200")   # alternative constructor

acc1.deposit(250)
acc1.withdraw(100)
print(acc1.owner, acc1.get_balance())          # public access + controlled access
print(acc2.owner, acc2.get_balance())

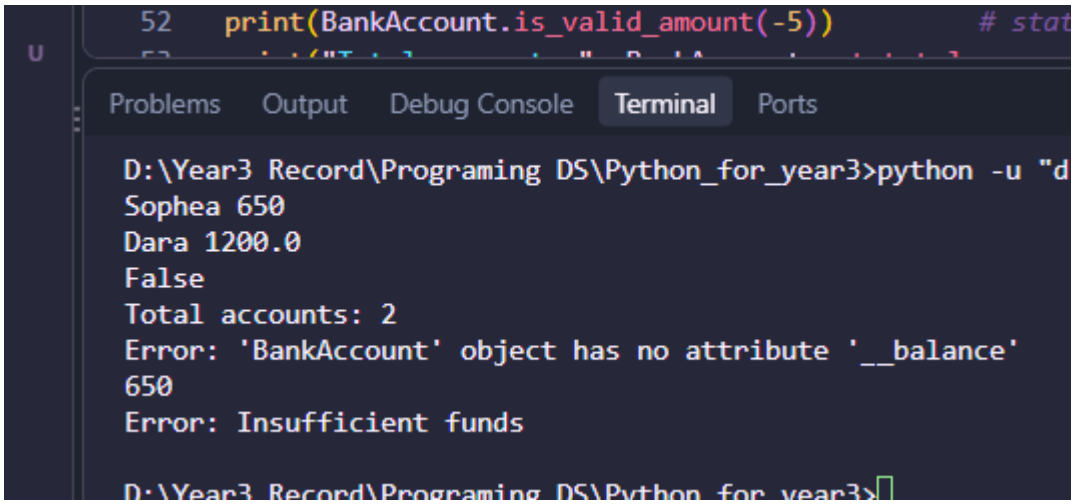
print(BankAccount.is_valid_amount(-5))         # static method
print("Total accounts:", BankAccount.get_total_accounts())   # class method

# Trying to bypass the interface
try:
    print(acc1.__balance)
except AttributeError as e:
    print("Error:", e)

print(acc1._BankAccount__balance)              # mangled name still works

# Business rule enforced by the interface
try:
    acc1.withdraw(5000)
except ValueError as e:
    print("Error:", e)
```

➢ Output:



Key Takeaways

- name is public, _name is a convention for internal use, and __name is name-mangled to _ClassName__name.
- Python relies on convention and trust, not strict enforcement.
- Keep sensitive data (like a balance) private and expose it through methods that enforce rules.
- Use @staticmethod for helpers that need no class or instance data.
- Use @classmethod when you need the class itself, especially for alternative constructors and class-wide data.

Group 4: Python Modifiers and OOP Features — Part 2
e. Abstract Classes and Methods (abc, @abstractmethod)

- An **abstract class** defines a common interface that subclasses must follow. It cannot be instantiated directly. Python provides this through the abc module (Abstract Base Classes).

```
TP1 > test.py > ...
1   from abc import ABC, abstractmethod
2
3   class Shape(ABC):
4       @abstractmethod
5       def area(self):
6           """Every subclass must implement this."""
7           pass
8
9   s = Shape()          # TypeError: Can't instantiate abstract class Shape

Problems  Output  Debug Console  Terminal  Ports

D:\Year3 Record\Programing DS\Python_for_year3>python -u "d:\Year3 Record\Programing DS\Python_for_year
Traceback (most recent call last):
  File "d:\Year3 Record\Programing DS\Python_for_year3\TP1\test.py", line 9, in <module>
    s = Shape()          # TypeError: Can't instantiate abstract class Shape
TypeError: Can't instantiate abstract class Shape without an implementation for abstract method 'area'
```

- A subclass must override **all** abstract methods, otherwise it is also abstract and cannot be instantiated

```
TP1 > test.py > ...
1   from abc import ABC, abstractmethod
2
3   class Shape(ABC):
4       @abstractmethod
5       def area(self):
6           """Every subclass must implement this."""
7           pass
8   class Square(Shape):
9       def __init__(self, side):
10          self.side = side
11
12      def area(self):
13          return self.side ** 2
14
15  sq = Square(4)
16  print(sq.area())      # 16
17

roblems  Output  Debug Console  Terminal  Ports

D:\Year3 Record\Programing DS\Python_for_year3>python -u "d
16
```

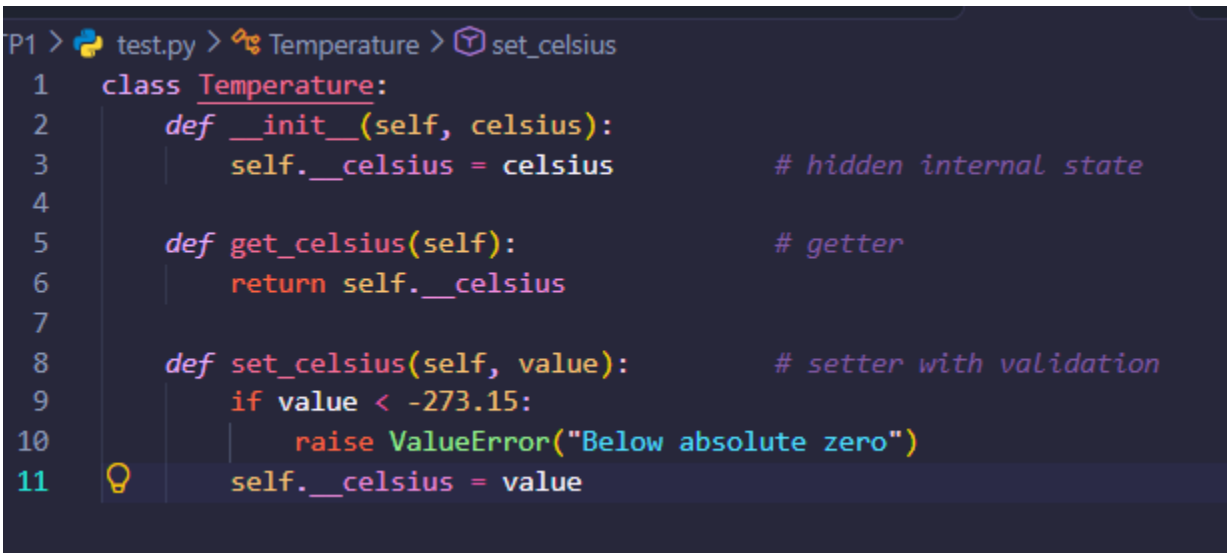
Key points:

- Inherit from ABC (or use metaclass=ABCMeta).
- An abstract class can also contain normal (concrete) methods and attributes that subclasses inherit.
- Abstract properties are possible by stacking @property on top of @abstractmethod.
- Abstract classes enforce a contract: "any Shape must be able to compute its area."

f. Access Control and Encapsulation

Encapsulation means bundling data with the methods that work on it, and controlling how that data is accessed. In Python this is done by:

1. Using naming conventions (_name, __name) to mark internal data.
2. Exposing a clean public interface (methods or properties).
3. Validating data before it is stored.



- **Benefits:** data stays valid, internal details can change without breaking outside code, and the class stays easy to use. The `@property` decorator (next section) makes this approach more Pythonic.

g. Other OOP Features: @property, @final, and typing.final

@property

- @property lets you access a method like an attribute, while still running validation or computation.

```python
class Temperature:
    def __init__(self, celsius):
        self.celsius = celsius              # goes through the setter

    @property
    def celsius(self):                      # getter
        return self.__celsius

    @celsius.setter
    def celsius(self, value):               # setter
        if value < -273.15:
            raise ValueError("Below absolute zero")
        self.__celsius = value

    @property
    def fahrenheit(self):                   # read-only computed property
        return self.__celsius * 9 / 5 + 32

t = Temperature(25)
print(t.celsius)          # 25 (looks like an attribute)
print(t.fahrenheit)       # 77.0
t.celsius = 30            # runs the setter
t.fahrenheit = 100        # AttributeError: no setter defined (read-only)
```

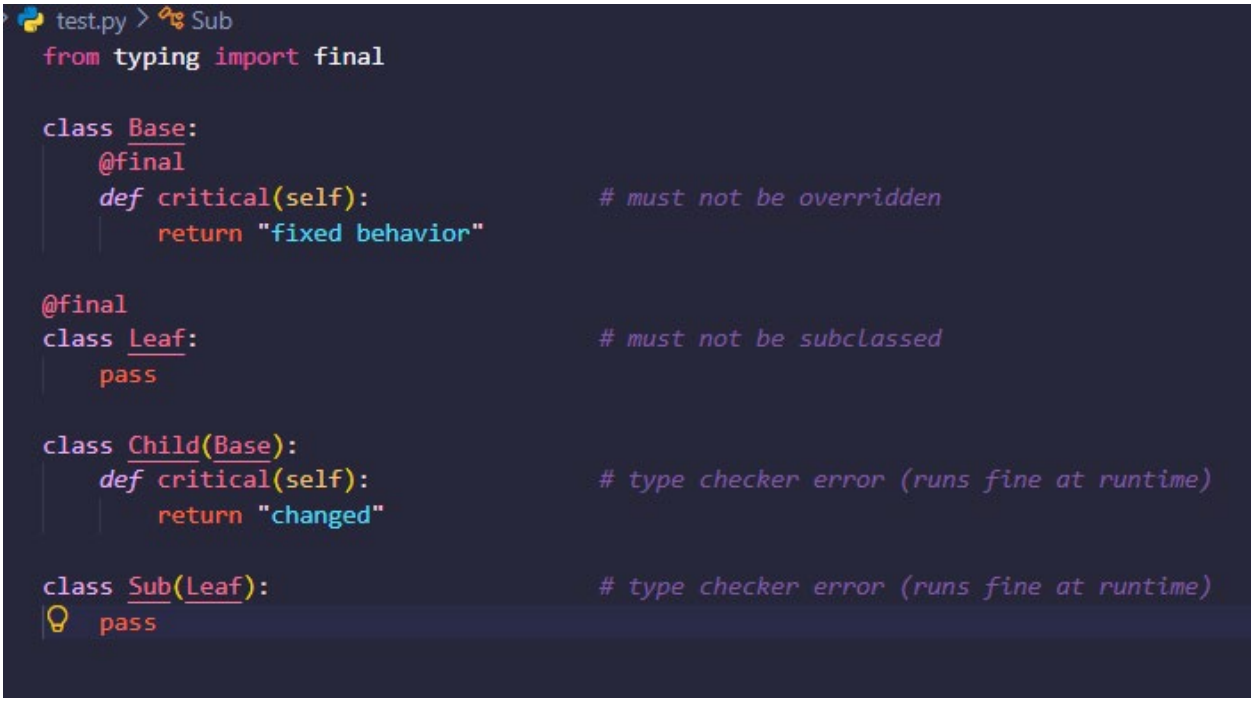
```
D:\Year3 Record\Programing DS\Python for year3>python -u "d:\Year3 Record\Programing DS\Pyth
25
77.0
Traceback (most recent call last):
  File "d:\Year3 Record\Programing DS\Python_for_year3\TP1\test.py", line 23, in <module>
    t.fahrenheit = 100        # AttributeError: no setter defined (read-only)
    ^^^^^^^^^^^^
AttributeError: property 'fahrenheit' of 'Temperature' object has no setter

D:\Year3 Record\Programing DS\Python_for_year3>
```

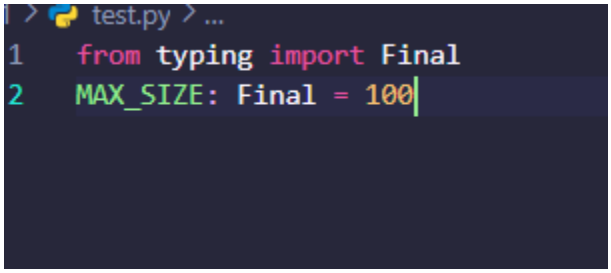
- **Why use it?** You can start with a plain attribute and add validation later **without changing the calling code**.

@final and typing.final

- Python has no built-in final keyword like Java. Since Python 3.8, typing.final marks that something should **not be overridden or subclassed**. It is mainly a signal for type checkers (such as mypy) and, since Python 3.11, it also sets a __final__ attribute. It is **not enforced at runtime**.



- There is also `typing.Final` (capital F) for constants that should not be reassigned:



Comparison:

Feature	*Purpose*	*Enforced at runtime?*
ABC + @abstractmethod	Force subclasses to implement methods	Yes (on instantiation)
_name / __name	Signal or mangle internal data	Convention / mangling only
@property	Controlled attribute-style access	Yes
@final / Final	Prevent overriding, subclassing, or reassignment	No (type checkers only)

Practical example: an abstract Shape class with Circle and Rectangle subclasses.

```python
from abc import ABC, abstractmethod
import math
from typing import final

class Shape(ABC):
    def __init__(self, name):
        self._name = name                       # non-public attribute

    @property
    def name(self):                             # read-only public access
        return self._name

    @abstractmethod
    def area(self):
        """Return the area of the shape."""

    @abstractmethod
    def perimeter(self):
        """Return the perimeter of the shape."""

    @final
    def describe(self):                         # concrete method, must not be overridden
        return (f"{self.name}: area = {self.area():.2f}, "
                f"perimeter = {self.perimeter():.2f}")
```

```python
class Circle(Shape):
    def __init__(self, radius):
        super().__init__("Circle")
        self.radius = radius                    # uses the property setter

    @property
    def radius(self):
        return self.__radius

    @radius.setter
    def radius(self, value):
        if value <= 0:
            raise ValueError("Radius must be positive")
        self.__radius = value

    def area(self):
        return math.pi * self.__radius ** 2

    def perimeter(self):
        return 2 * math.pi * self.__radius
```

```python

class Rectangle(Shape):
    def __init__(self, width, height):
        super().__init__("Rectangle")
        self.width = width
        self.height = height

    @property
    def width(self):
        return self.__width

    @width.setter
    def width(self, value):
        if value <= 0:
            raise ValueError("Width must be positive")
        self.__width = value

    @property
    def height(self):
        return self.__height

    @height.setter
    def height(self, value):
        if value <= 0:
            raise ValueError("Height must be positive")
        self.__height = value

```

```python
    def area(self):
        return self.__width * self.__height

    def perimeter(self):
        return 2 * (self.__width + self.__height)


# Using the classes
try:
    shape = Shape("Generic")                    # abstract: cannot instantiate
except TypeError as e:
    print("Error:", e)

shapes = [Circle(3), Rectangle(4, 5)]           # polymorphism: one list, many shapes

for s in shapes:
    print(s.describe())
```

```python
c = shapes[0]
c.radius = 5                                    # valid update through the setter
print(f"New area: {c.area():.2f}")

try:
    c.radius = -2                               # validation blocks bad data
except ValueError as e:
    print("Error:", e)
```

- Output:

```
Problems  Output  Debug Console  Terminal  Ports

D:\Year3 Record\Programing DS\Python_for_year3>python -u "d:\Year3 Record\Programing DS\Python_for_year3\TP1\test.py"
Error: Can't instantiate abstract class Shape without an implementation for abstract methods 'area', 'perimeter'
Circle: area = 28.27, perimeter = 18.85
Rectangle: area = 20.00, perimeter = 18.00
New area: 78.54
Error: Radius must be positive
```

Key Takeaways

- **Abstract classes** (ABC + @abstractmethod) define a contract that every subclass must fulfill, and cannot be instantiated themselves.

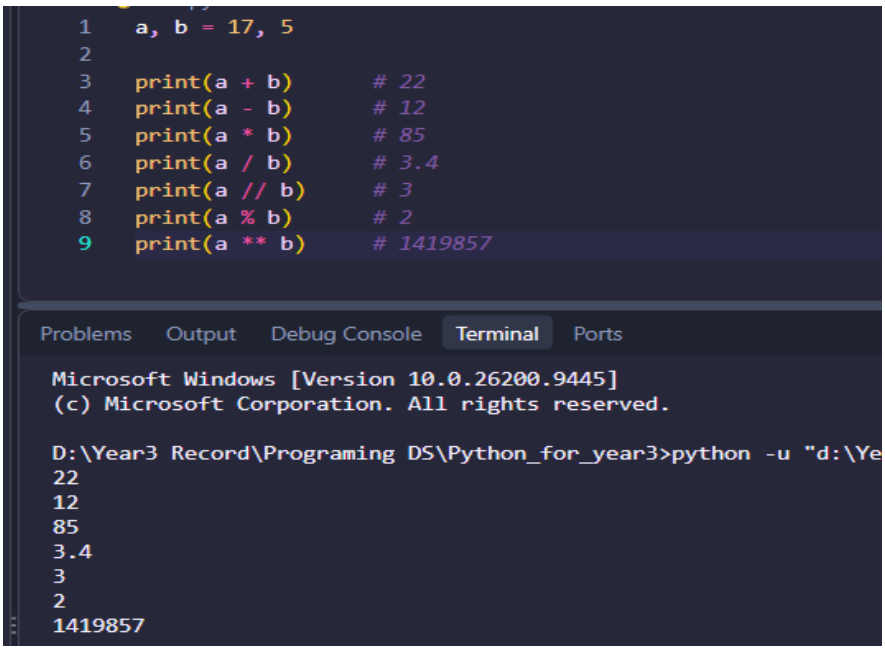
- **Encapsulation** hides internal data (_x, __x) and exposes a controlled public interface.
- **@property** gives attribute-style access with validation, and a property without a setter is read-only.
- **@final** documents and type-checks that a method or class should not be overridden or extended, but Python itself does not enforce it.
- **Polymorphism:** code like s.describe() works for any Shape subclass without knowing which one it is.

Group 5: Python Operators — Part 1

a. Arithmetic Operators (`+`, `-`, `*`, `/`, `//`, `%`, `**`)

Operator	Name	Example	Result
+	Addition	7 + 3	10
-	Subtraction	7 - 3	4
*	Multiplication	7 * 3	21
/	True division (always float)	7 / 2	3.5
//	Floor division	7 // 2	3
%	Modulus (remainder)	7 % 2	1
**	Exponent (power)	2 ** 5	32

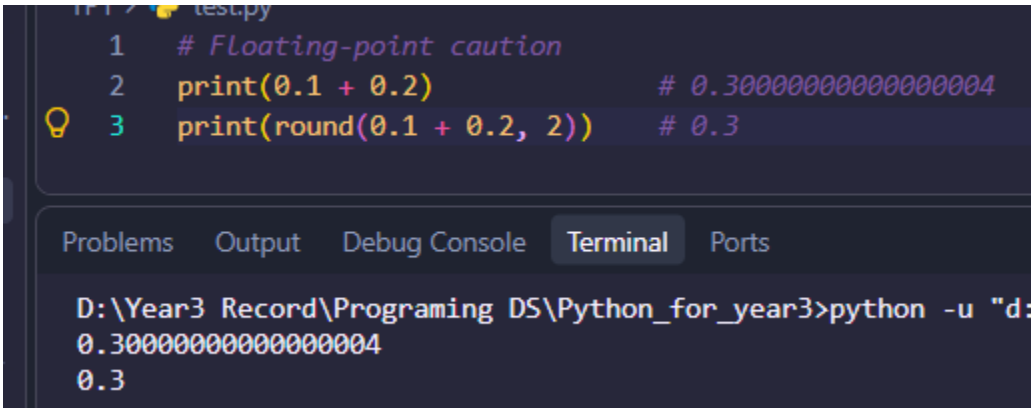
Example:



Key points:

- / always returns a float, even for 10 / 2 (which gives 5.0).
- // rounds **down** (toward negative infinity), so -7 // 2 is -4, not -3.
- % takes the sign of the divisor: -7 % 3 is 2.
- divmod(a, b) returns both results at once: divmod(17, 5) gives (3, 2).
- Precedence: ** first, then * / // %, then + -. Use parentheses to be clear.
- + and * also work on strings and lists: "ab" * 3 gives "ababab".

Augmented assignment: x += 5, x -= 2, x *= 3, x //= 2, x %= 4, x **= 2.



b. Comparison Operators (==, !=, >, <, >=, <=)

- They compare two values and return bool.

Operator	Meaning	Example	Result
==	Equal to	5 == 5	True
!=	Not equal to	5 != 3	True
>	Greater than	5 > 3	True
<	Less than	5 < 3	False
>=	Greater than or equal	5 >= 5	True
<=	Less than or equal	4 <= 3	False

Example:

```python
score = 72

print(score == 72)        # True
print(score != 100)       # True
print(score >= 50)        # True
print(score < 50)         # False

# Chained comparison (a Python feature)
print(50 <= score <= 100)     # True, same as (50 <= score) and (score <= 100)

# Works on strings (alphabetical order) and lists too
print("apple" < "banana")     # True
print([1, 2] == [1, 2])       # True
```

```
Microsoft Windows [Version 10.0.26200.9445]
(c) Microsoft Corporation. All rights reserved.

D:\Year3 Record\Programing DS\Python_for_year3>python -u "d:\Year3 Record\Programing DS
True
True
True
False
True
True
True
```

Common mistake: = assigns a value, == compares values.

Also remember: == checks equal **values**, while is checks the **same object**.

```python
a = [1, 2]
b = [1, 2]
print(a == b)      # True  (same contents)
print(a is b)      # False (different objects)
```

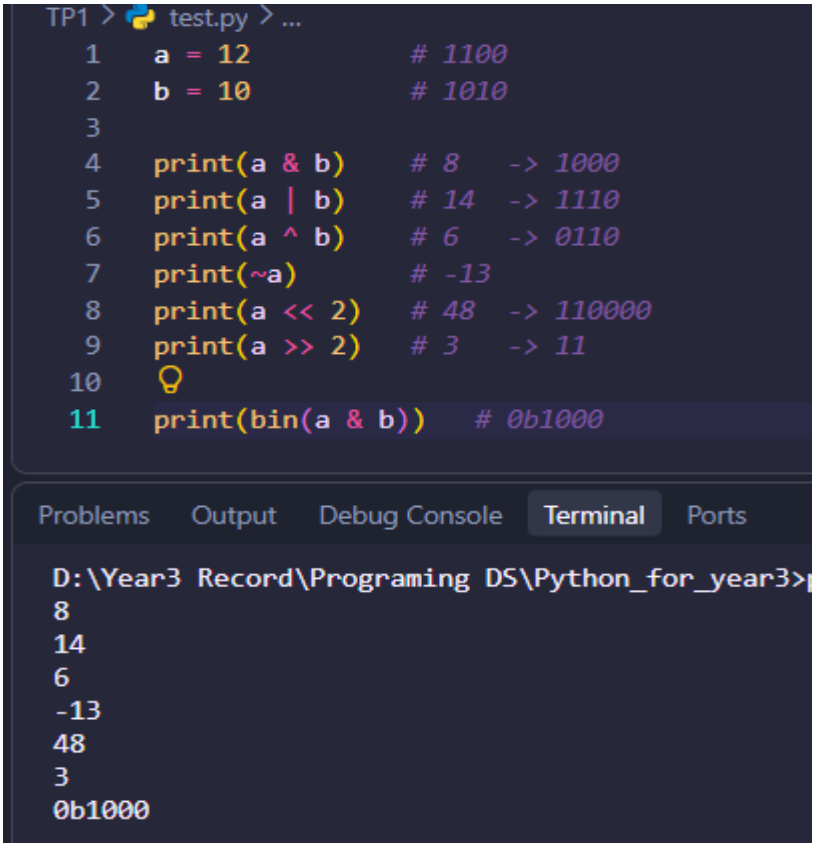
```
D:\Year3 Record\Programing DS\Python_for_year3>python
True
False
```

c. Bitwise Operators (`&`, `|`, `^`, `~`, `<<`, `>>`)

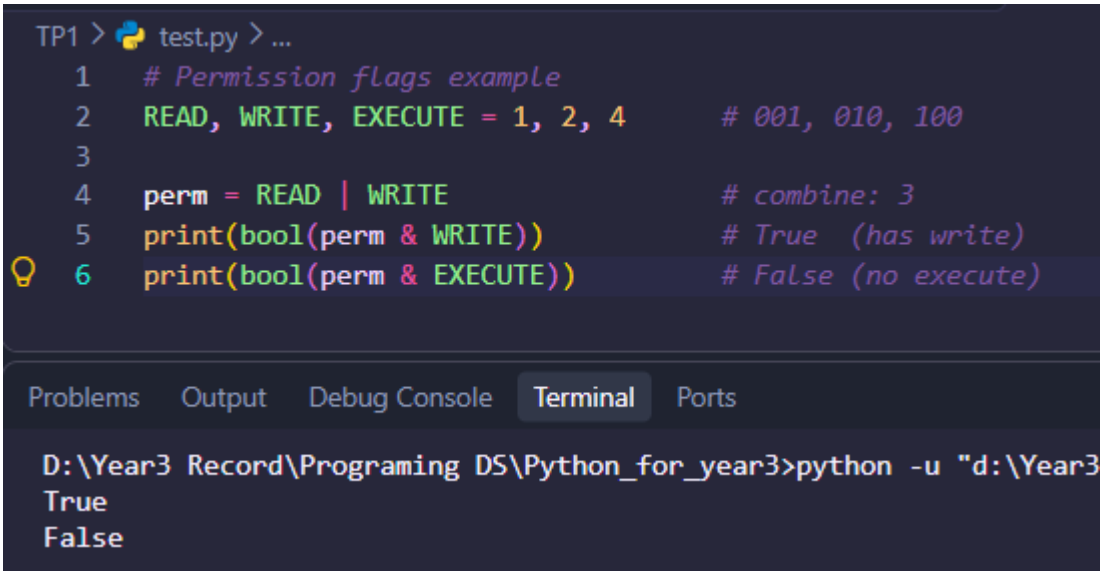
- These work on the **binary digits** of integers.

Operator	*Name*	*Rule*	*Example (a=12, b=10)*
&	AND	1 if both bits are 1	12 & 10 gives 8
|	OR	1 if either bit is 1	12 | 10 gives 14
^	XOR	1 if bits differ	12 ^ 10 gives 6
~	NOT	Flips all bits: ~x = -x – 1	~12 gives -13
<<	Left shift	Multiply by 2 per shift	12 << 2 gives 48
>>	Right shift	Floor-divide by 2 per shift	12 >> 2 gives 3

Example:



Typical uses: permission flags, masks, low-level data, and fast multiplication or division by powers of 2.



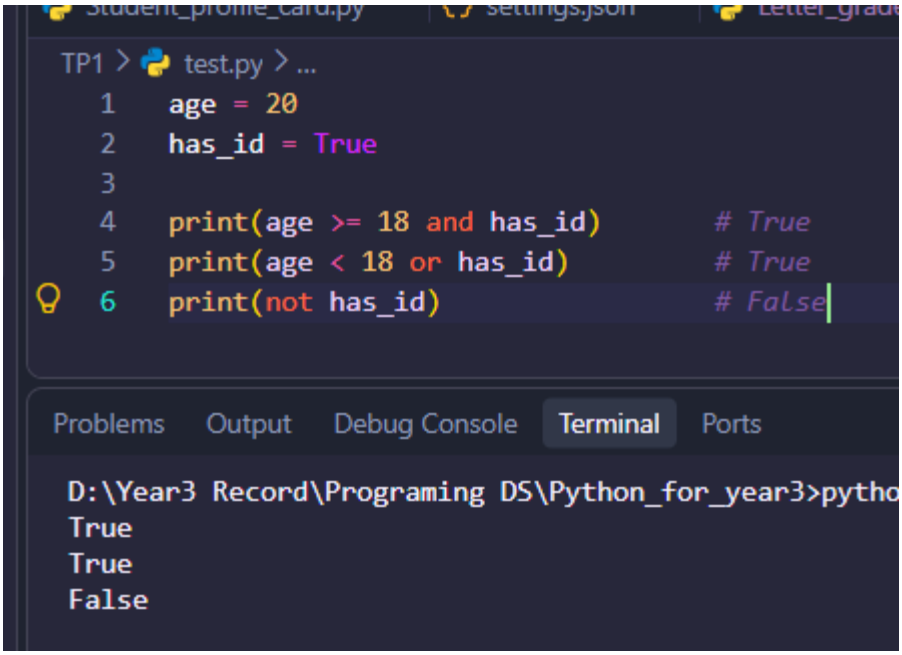
Note: Bitwise &, | are different from logical and, or.

d. Logical Operators (`and`, `or`, `not`)

Operate	Meaning	Example	Result
and	True only if both are true	True and False	False
or	True if at least one is true	True or False	True
not	Reverses the value	not True	False

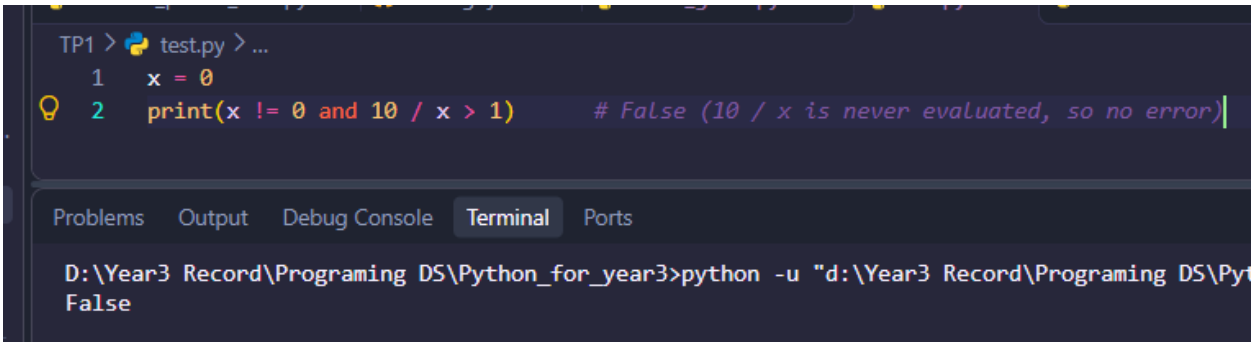
Truth table:

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

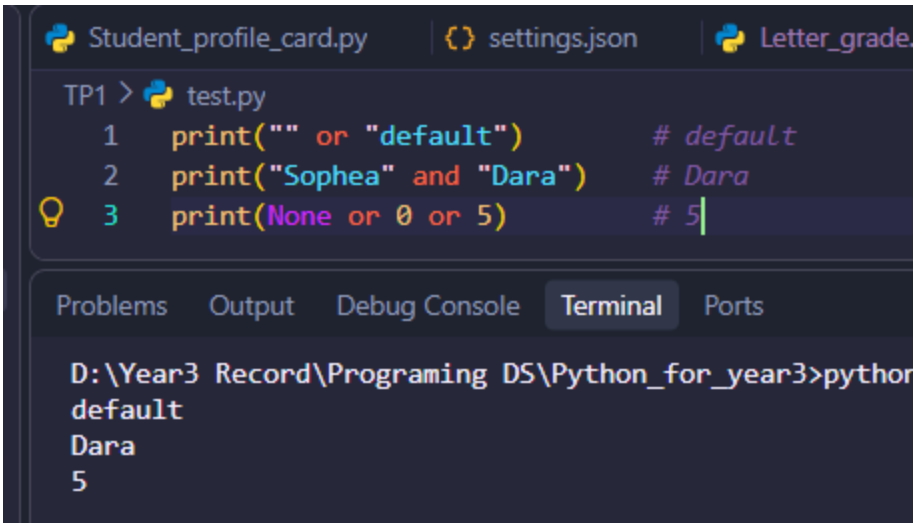


Short-circuit evaluation:

- and stops at the first falsy value.
- or stops at the first truthy value.



Logical operators return actual values, not just True/False:



Falsy values: False, None, 0, 0.0, "", [], (), {}, set().

Precedence: not first, then and, then or.

- Practical example: calculating student grades and checking eligibility.

```python
# Student data
name = "Sophea"
midterm = 78
final = 85
project = 90
attendance = 18      # classes attended
total_classes = 20

# ARITHMETIC: calculate weighted score and attendance rate
total_score = midterm * 0.30 + final * 0.50 + project * 0.20
attendance_rate = attendance / total_classes * 100
bonus_points = total_score // 10          # whole tens in the score
remainder = total_score % 10              # leftover points
score_squared = 5 ** 2                    # 25, example of **

print(f"Total score: {total_score:.1f}")
print(f"Attendance: {attendance_rate:.0f}%")
```

```
    print(f"Total score: {total_score:.1f}")
    print(f"Attendance: {attendance_rate:.0f}%")

    # COMPARISON: grade decision
    is_passing = total_score >= 50
    is_top = total_score >= 85
    good_attendance = attendance_rate >= 80
    print("Passing:", is_passing)

    # Grade letter using comparisons and chaining
    if total_score >= 85:
        grade = "A"
    elif 70 <= total_score < 85:
        grade = "B"
    elif 50 <= total_score < 70:
        grade = "C"
    else:
        grade = "F"
```

```
        grade = "F"
    print("Grade:", grade)

    # LOGICAL: eligibility checks
    eligible_for_exam = is_passing and good_attendance
    eligible_for_scholarship = total_score >= 80 and attendance_rate >= 90 and project >= 85
    needs_support = not is_passing or not good_attendance
    print("Eligible for next level:", eligible_for_exam)
    print("Scholarship:", eligible_for_scholarship)
    print("Needs support:", needs_support)

    # BITWISE: store achievements as flags
    HONOR_ROLL = 1      # 001
    PERFECT_ATT = 2     # 010
    TOP_PROJECT = 4     # 100

    badges = 0
    if total_score >= 80:
        badges |= HONOR_ROLL            # add a badge
    if attendance == total_classes:
        badges |= PERFECT_ATT
    if project >= 90:
        badges |= TOP_PROJECT

    print("Badges value:", badges, bin(badges))
    print("Has honor roll:", bool(badges & HONOR_ROLL))
    print("Has perfect attendance:", bool(badges & PERFECT_ATT))
    badges &= ~TOP_PROJECT              # remove a badge
    print("After removing top project:", badges)
```

Output:

```
D:\Year3 Record\Programing DS\Python_for_year3>pyt
Total score: 83.9
Attendance: 90%
Passing: True
Grade: B
Eligible for next level: True
Scholarship: True
Needs support: False
Badges value: 5 0b101
Has honor roll: True
Has perfect attendance: False
After removing top project: 1
```

Key Takeaways

- Arithmetic: / gives a float, // floors, % gives the remainder, ** is power.
- Comparison operators return bool and can be chained (50 <= x <= 100).
- Bitwise operators work on binary digits and are useful for flags and masks.
- Logical operators (and, or, not) short-circuit and return actual operand values.
- Do not confuse = with ==, is with ==, or &/| with and/or.

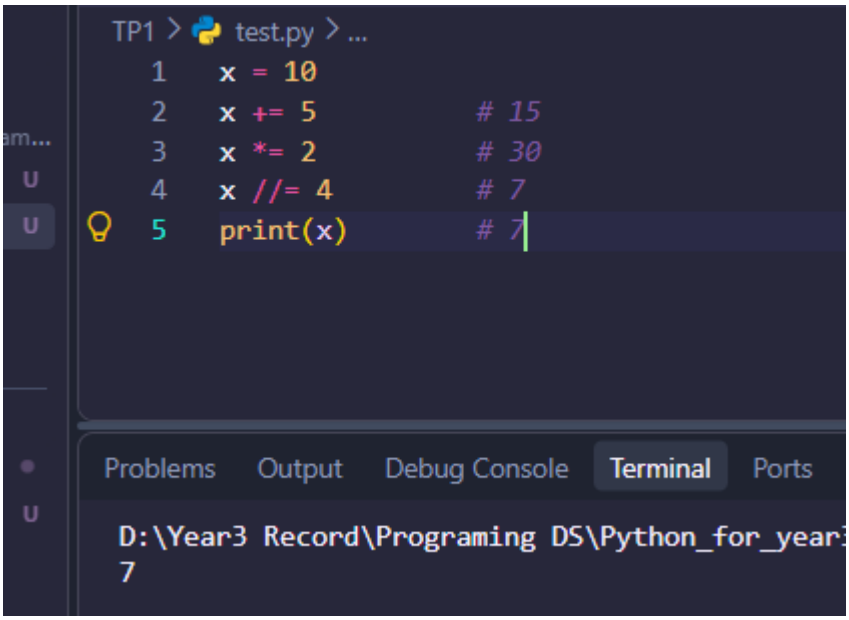
Group 6: Python Operators — Part 2
e. Assignment Operators (=, +=, -=, *=, etc.)

- The basic = binds a name to a value. **Augmented assignment** operators combine an operation with assignment.

Operator	Equivalent to	Example (x = 10)	Result
=	Assign	x = 10	10
+=	x = x + n	x += 5	15

-=	x = x - n	x -= 5	7
/=	x = x / n	x /= 4	2.5
//=	x = x // n	x //= 3	3
%=	x = x % n	x %= 3	1
**=	x = x ** n	x **= 2	100
&=, |=, ^=	bitwise + assign	x &= 6	2
*=	x = x * n	x *= 2	20
<<=, >>=	Shift + assgin	x <<= 1	20

Example:



f. Special Operators: Identity (is, is not), Membership (in, not in), and Conditional Expressions (x if condition else y)

- **Identity:** `is`, `is not`
- These check whether two names refer to the **same object** (same memory identity), not just equal values.

```
TP1 > test.py > ...
1   a = [1, 2, 3]
2   b = a                # same object
3   c = [1, 2, 3]        # equal, but a different object
4
5   print(a is b)        # True
6   print(a is c)        # False
7   print(a == c)        # True  (values are equal)
8   print(a is not c)    # True
9   print(id(a) == id(b))   # True, is compares id()
```

```
Problems  Output  Debug Console  Terminal  Ports

D:\Year3 Record\Programing DS\Python_for_year3>python -u "d:\Ye
True
False
True
True
True
```

- **Best practice:** use `is` for `None` (and `True`/`False` singletons), and `==` for comparing values.

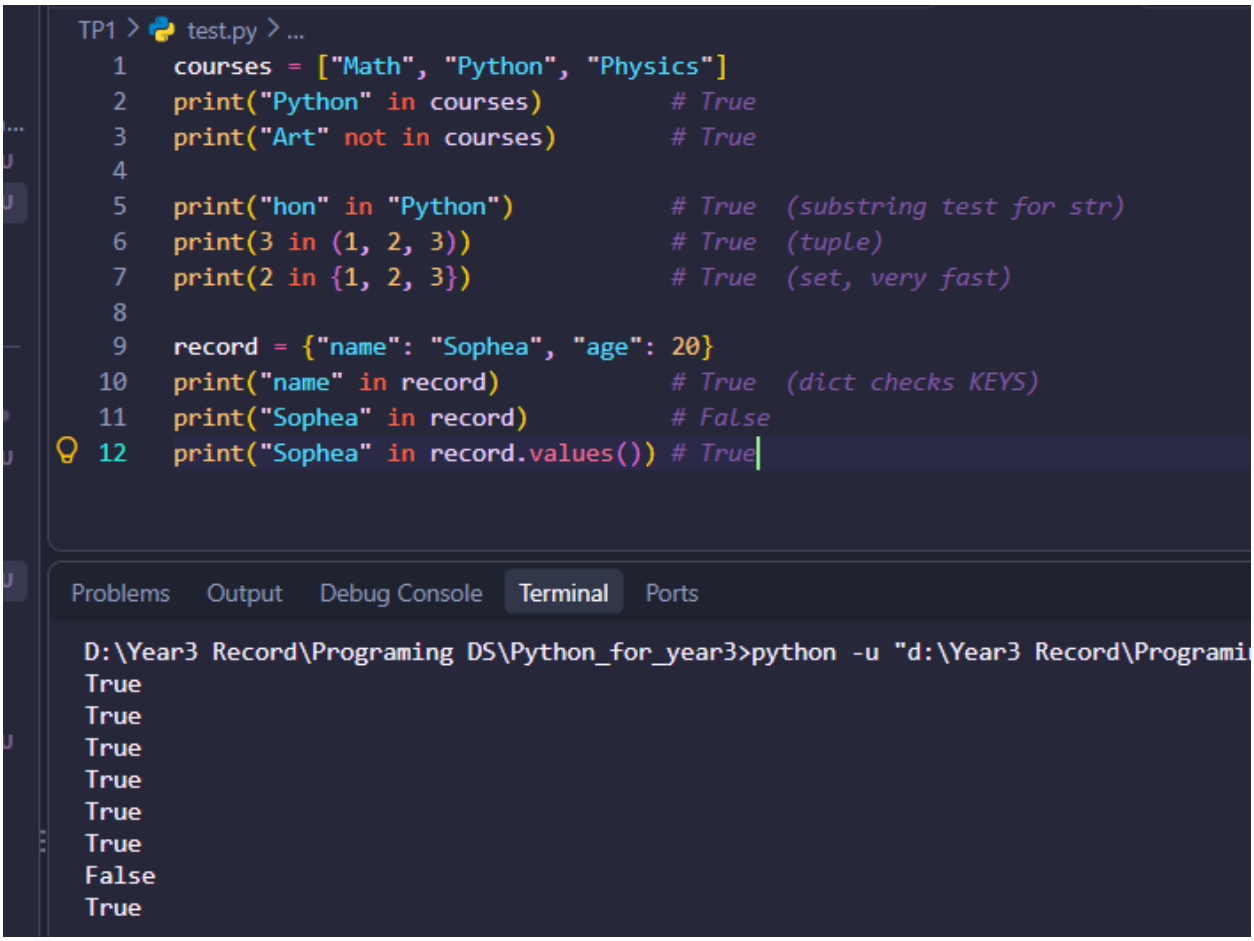
```
TP1 > test.py > ...
1   result = None
2   if result is None:          # correct way
3       print("No result yet")
```

```
Problems  Output  Debug Console  Terminal  Ports

D:\Year3 Record\Programing DS\Python_for_year3>python
No result yet
```

- **Caution:** do not use `is` to compare numbers or strings. Small ints and some strings may be cached, so the outcome can vary and should not be relied on.

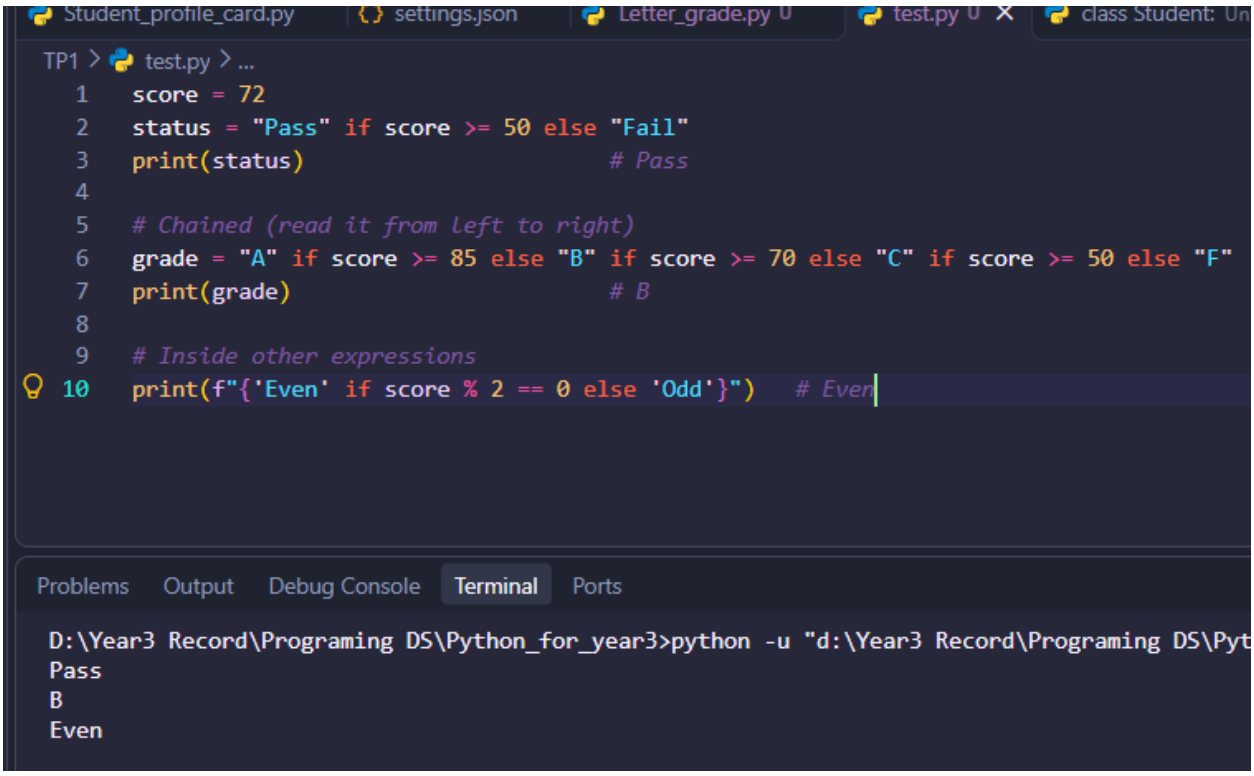
- **Membership:** `in`, `not in`
- These test whether a value exists inside a sequence or collection.



Performance tip: membership in a set or dict is very fast (about constant time), while a list or tuple is scanned item by item.

Conditional Expression: x if condition else y

A one-line version of if/else that produces a value (also called the ternary operator).



Use it for simple choices between two values. For complex logic, a regular if/elif/else is easier to read.

g. Operator Precedence and Associativity

- **Precedence** decides which operator is evaluated first. **Associativity** decides the order when operators have the same precedence.

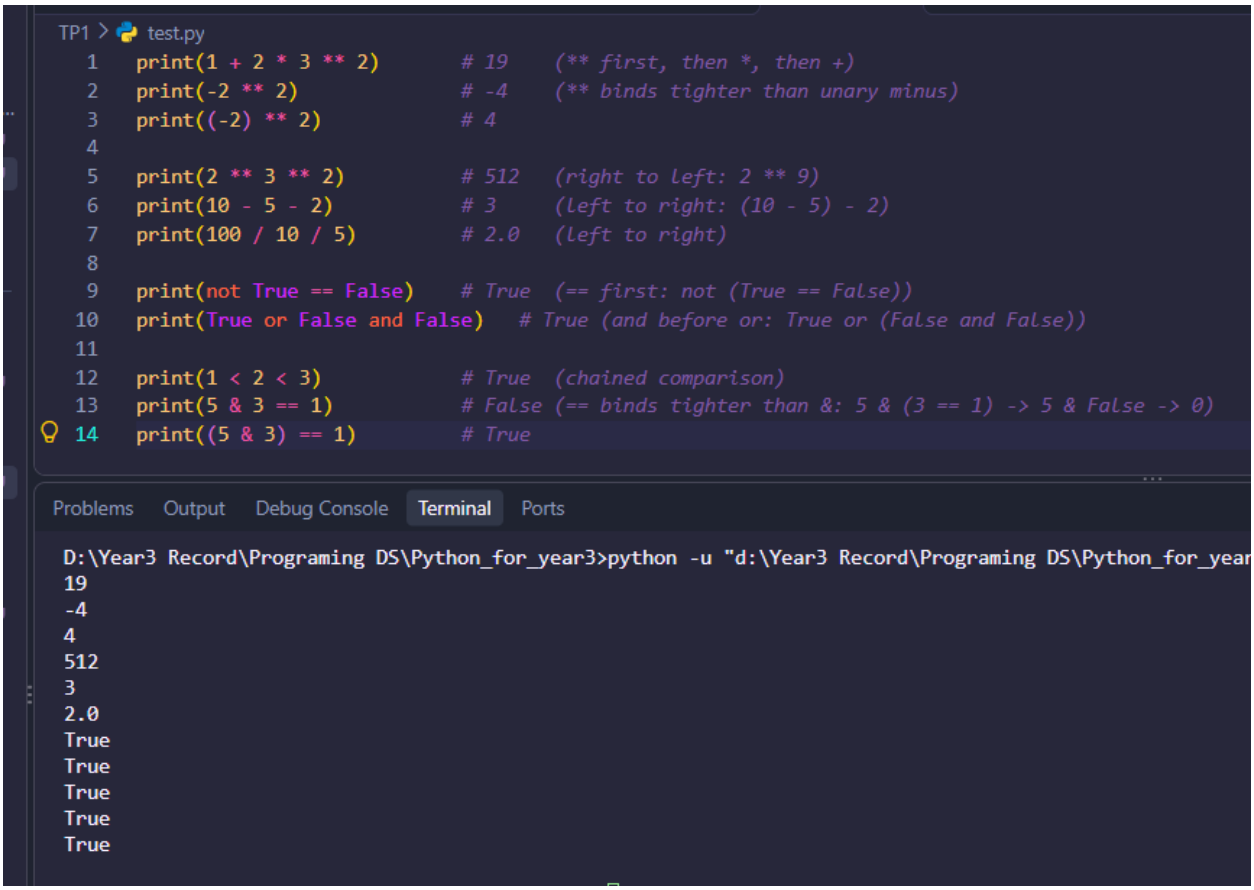
Precedence table (highest to lowest)

Level	Operators	Associativity
1	( ) parentheses	n/a
2	**x[i], x.attr, f()**	left to right
3	**	right to left
4	+x, -x, ~x (unary)	right to left
5	*, /, //, %, @	left to right
6	+, -	left to right

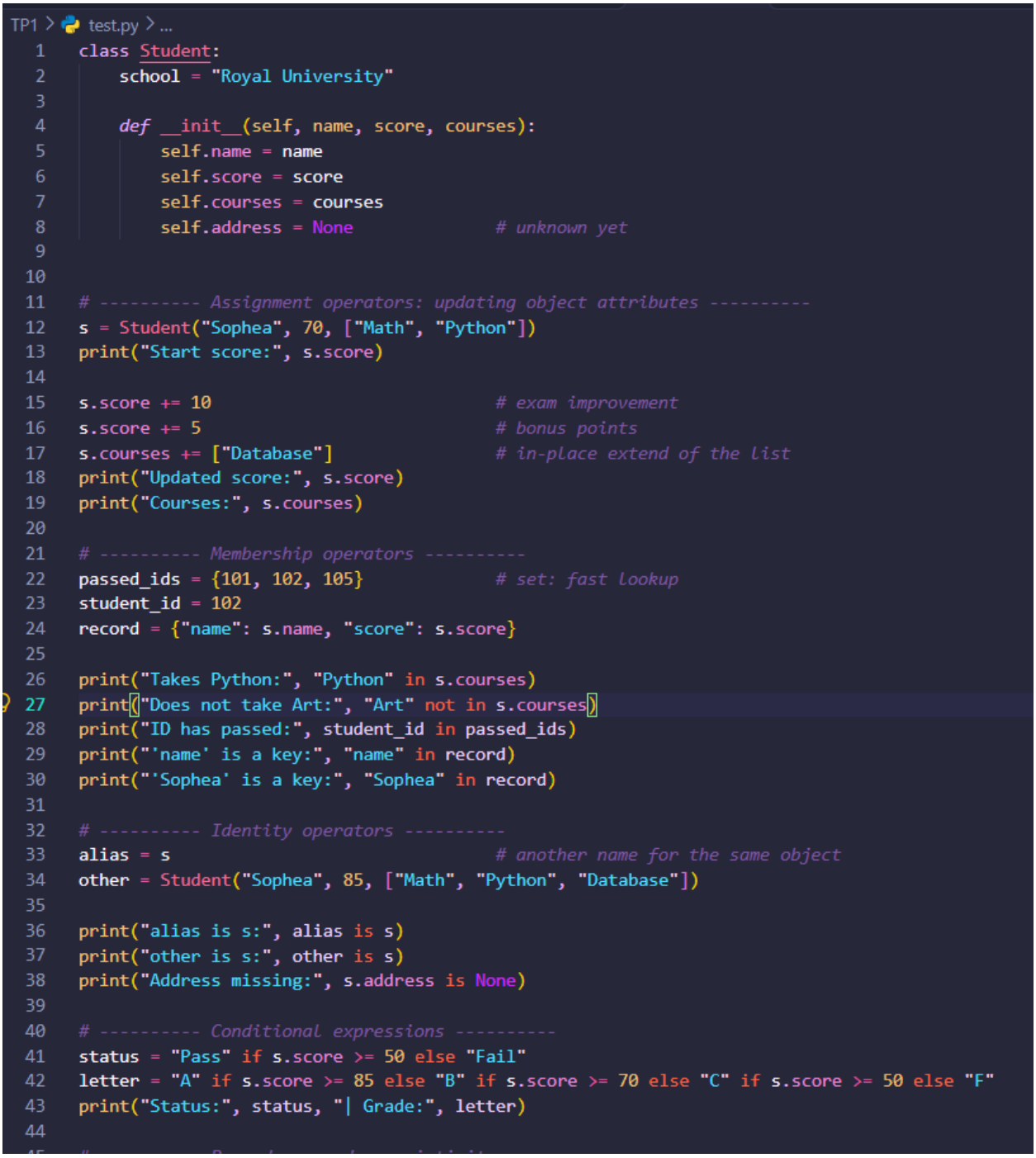
7	<<, >>	left to right
8	&	left to right
9	^	left to right
10	|	left to right
11	==, !=, <, >, <=, >=, is, is not, in, not in	chained
12	not	right to left
13	and	left to right
14	or	left to right
15	x if c else y	right to left
16	Lambda	n/a
17	:=	n/a

Assignment (=, +=, ...) is a statement, not an operator, and it evaluates the right side first.

Example:



Practical example: updating object attributes, checking membership in collections, and evaluating expressions.



```python
class Student:
    school = "Royal University"

    def __init__(self, name, score, courses):
        self.name = name
        self.score = score
        self.courses = courses
        self.address = None             # unknown yet


# ---------- Assignment operators: updating object attributes ----------
s = Student("Sophea", 70, ["Math", "Python"])
print("Start score:", s.score)

s.score += 10                           # exam improvement
s.score += 5                            # bonus points
s.courses += ["Database"]               # in-place extend of the list
print("Updated score:", s.score)
print("Courses:", s.courses)

# ---------- Membership operators ----------
passed_ids = {101, 102, 105}            # set: fast lookup
student_id = 102
record = {"name": s.name, "score": s.score}

print("Takes Python:", "Python" in s.courses)
print("Does not take Art:", "Art" not in s.courses)
print("ID has passed:", student_id in passed_ids)
print("'name' is a key:", "name" in record)
print("'Sophea' is a key:", "Sophea" in record)

# ---------- Identity operators ----------
alias = s                               # another name for the same object
other = Student("Sophea", 85, ["Math", "Python", "Database"])

print("alias is s:", alias is s)
print("other is s:", other is s)
print("Address missing:", s.address is None)

# ---------- Conditional expressions ----------
status = "Pass" if s.score >= 50 else "Fail"
letter = "A" if s.score >= 85 else "B" if s.score >= 70 else "C" if s.score >= 50 else "F"
print("Status:", status, "| Grade:", letter)
```

```python
# ---------- Precedence and associativity ----------
expr1 = 2 + 3 * 4 ** 2 / 8 - 1
expr2 = 2 ** 3 ** 2
eligible = s.score >= 50 and "Python" in s.courses or s.name == "Admin"
print("expr1:", expr1)
print("expr2:", expr2)
print("Eligible:", eligible)
```

Output:

```
Problems  Output  Debug Console  Terminal  Ports

D:\Year3 Record\Programing DS\Python_for_year3>python -u "d:\Ye
Start score: 70
Updated score: 85
Courses: ['Math', 'Python', 'Database']
Takes Python: True
Does not take Art: True
ID has passed: True
'name' is a key: True
'Sophea' is a key: False
alias is s: True
other is s: False
Address missing: True
Status: Pass | Grade: A
expr1: 7.0
expr2: 512
Eligible: True
```

Key Takeaways

- **Augmented assignment** (+=, *=, ...) is shorthand, but on mutable objects such as lists it modifies the object **in place**.
- **is** compares object identity (use it for None), while == compares values.

- **in / not in** test membership, and for dictionaries they check the **keys**. Use sets for fast lookups.
- **x if condition else y** gives a compact one-line choice between two values.
- **Precedence:** ** is highest among arithmetic, then * / // %, then + -, then comparisons, then not, and, or. ** is **right-associative**, and most others are left-associative.
- Use parentheses whenever an expression could be misread.

Group 7: Python Decision-Making Statements

Decision-making statements let a program choose which block of code to run, depending on whether a condition is True or False.

Important rules:

- A condition ends with a colon :.
- The block under it must be **indented** (4 spaces is the standard).
- Any value can act as a condition: zero, empty collections, None, and "" count as False, while most other values count as True.

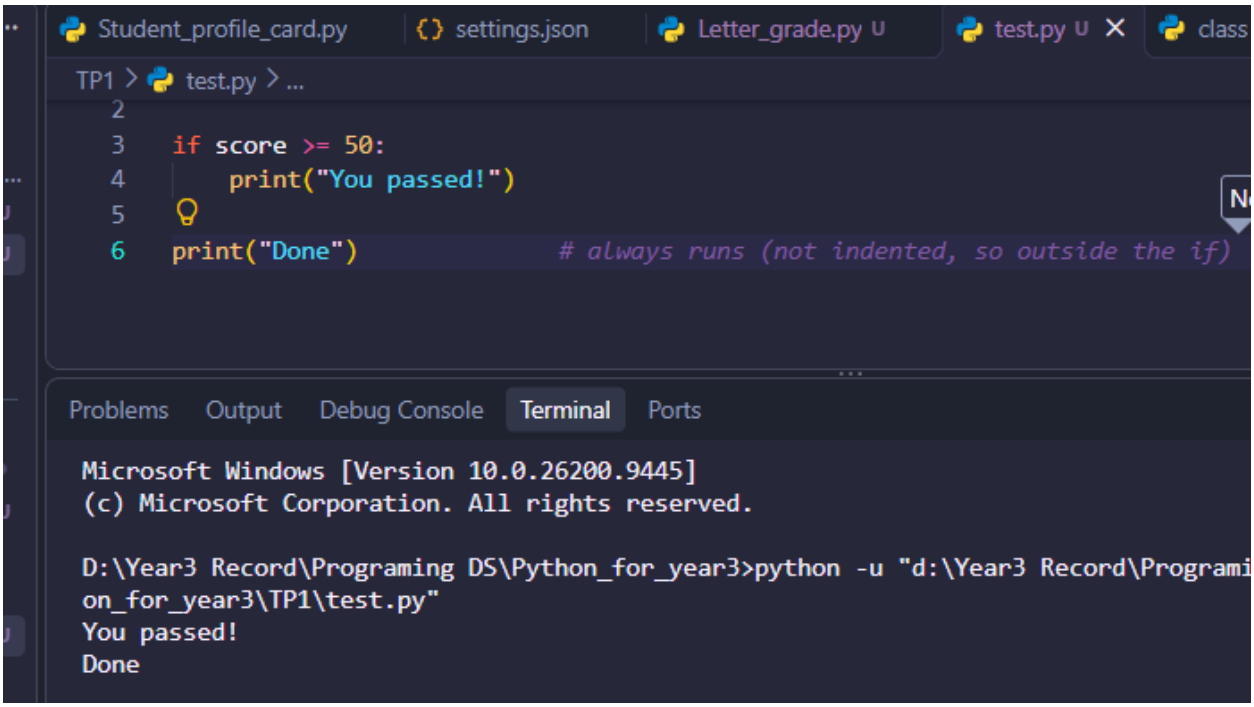
a. The if Statement

- Runs a block **only when** the condition is true. Otherwise, nothing happens.

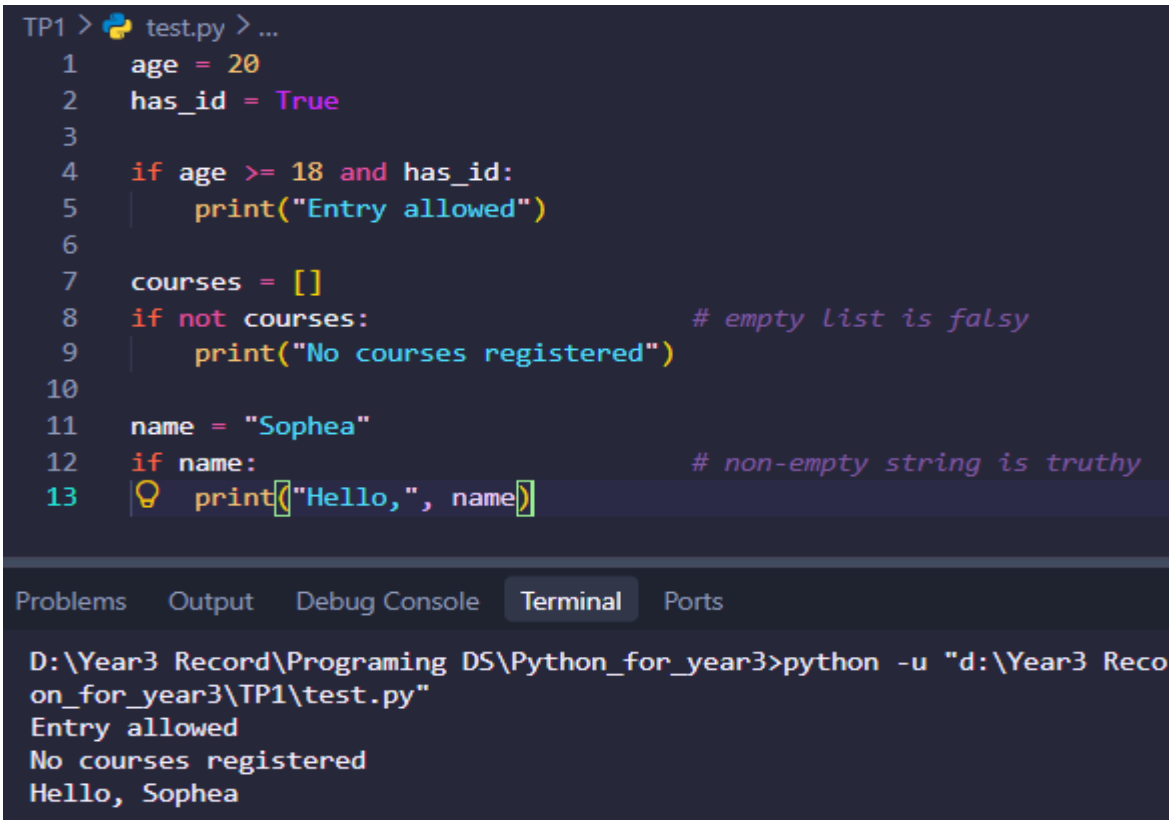
  Syntax:

  ```
  if condition:
      # runs only if condition is True
  ```

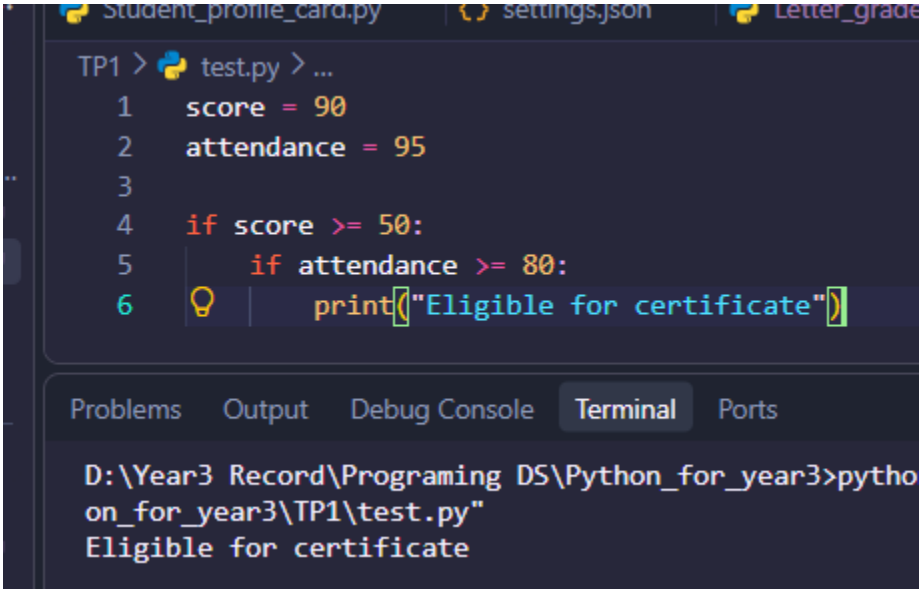
Example:



Combining conditions and truthiness:



Nested if (an if inside another if):



This is the same as if score >= 50 and attendance >= 80:, which is usually cleaner.

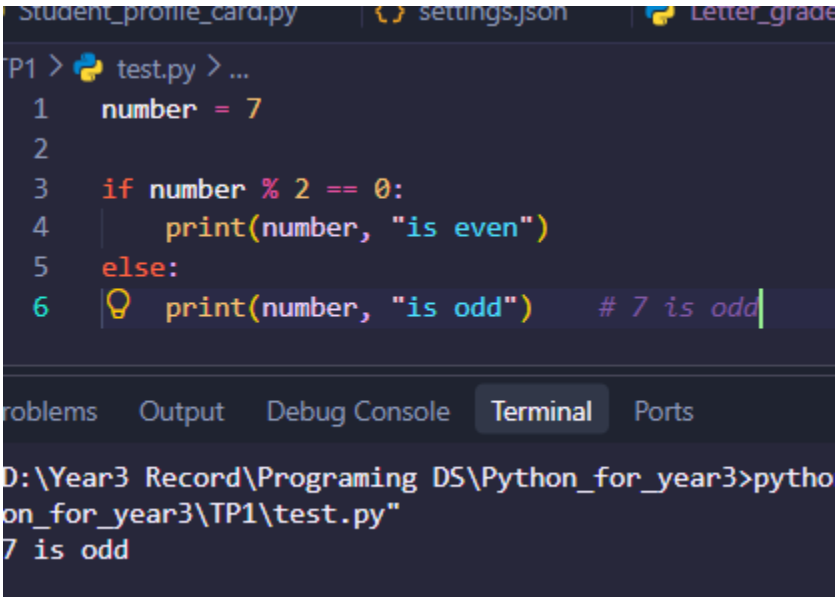
b. The if-else Statement

- Chooses between **two** paths. Exactly one block runs.

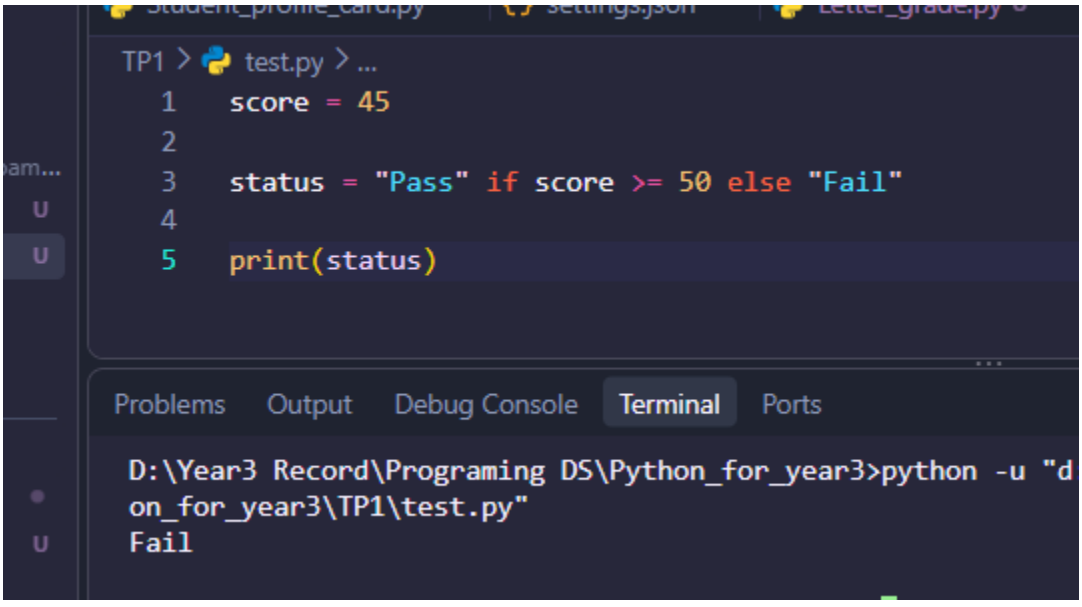
Syntax:

```
if condition:
    # runs if True
else:
    # runs if False
```

Example:



One-line alternative (conditional expression from Group 6):



c. The if-elif-else Statement

- Chooses among **many** paths. Python checks the conditions from top to bottom and runs the **first** block whose condition is true, then skips the rest.

Syntax:

if condition1:

block 1

elif condition2:

block 2

elif condition3:

block 3

else:

runs if none of the above were true

Example:

```
P1 > test.py > ...
1   score = 78
2
3   if score >= 85:
4       grade = "A"
5   elif score >= 70:
6       grade = "B"             # this runs; the remaining checks are skipped
7   elif score >= 50:
8       grade = "C"
9   else:
10      grade = "F"
11
12  print("Grade:", grade)       # Grade: B
```

```
roblems  Output  Debug Console  Terminal  Ports

D:\Year3 Record\Programing DS\Python_for_year3>python -u "d:\Year3 Record\Programi
on_for_year3\TP1\test.py"
Grade: B
```

Key points:

- elif means “else if”. You can have as many as you need.
- else is optional and must come last.
- Only **one** block in the whole chain runs.
- Separate if statements (instead of elif) are each checked independently.

Comparison

Statement	Number of paths	Runs when
if	1 (or nothing)	Condition is true
if – else	2	if block when true, else block otherwise
if – elif - else	3 or more	The first true condition, or else if none are ture

Practical example: determining student pass/fail status and assigning letter grades.

```python
test.py > ...
def evaluate_student(name, midterm, final, attendance_rate):
    # Weighted score
    total = midterm * 0.40 + final * 0.60

    # ----- if: simple check with a warning -----
    if attendance_rate < 60:
        print(f"Warning for {name}: attendance is very low ({attendance_rate}%)")

    # ----- if-else: pass or fail -----
    if total >= 50 and attendance_rate >= 75:
        status = "Pass"
    else:
        status = "Fail"

    # ----- if-elif-else: letter grade -----
    if total >= 90:
        grade = "A"
    elif total >= 80:
        grade = "B"
    elif total >= 70:
        grade = "C"
```

```
    elif total >= 80:
        grade = "B"
    elif total >= 70:
        grade = "C"
    elif total >= 60:
        grade = "D"
    elif total >= 50:
        grade = "E"
    else:
        grade = "F"

    # A student who fails because of attendance gets F regardless of score
    if status == "Fail":
        grade = "F"

    print(f"{name}: total = {total:.1f}, grade = {grade}, status = {status}")


evaluate_student("Sophea", 85, 92, 95)
evaluate_student("Dara", 70, 65, 80)
evaluate_student("Vanna", 40, 45, 90)
evaluate_student("Chenda", 88, 90, 55)
```

Output:

```
D:\Year3 Record\Programing DS\Python_for_year3>pyt
on_for_year3\TP1\test.py"
Sophea: total = 89.2, grade = B, status = Pass
Dara: total = 67.0, grade = D, status = Pass
Vanna: total = 43.0, grade = F, status = Fail
Warning for Chenda: attendance is very low (55%)
Chenda: total = 89.2, grade = F, status = Fail

D:\Year3 Record\Programing DS\Python_for_year3>
```

Key Takeaways

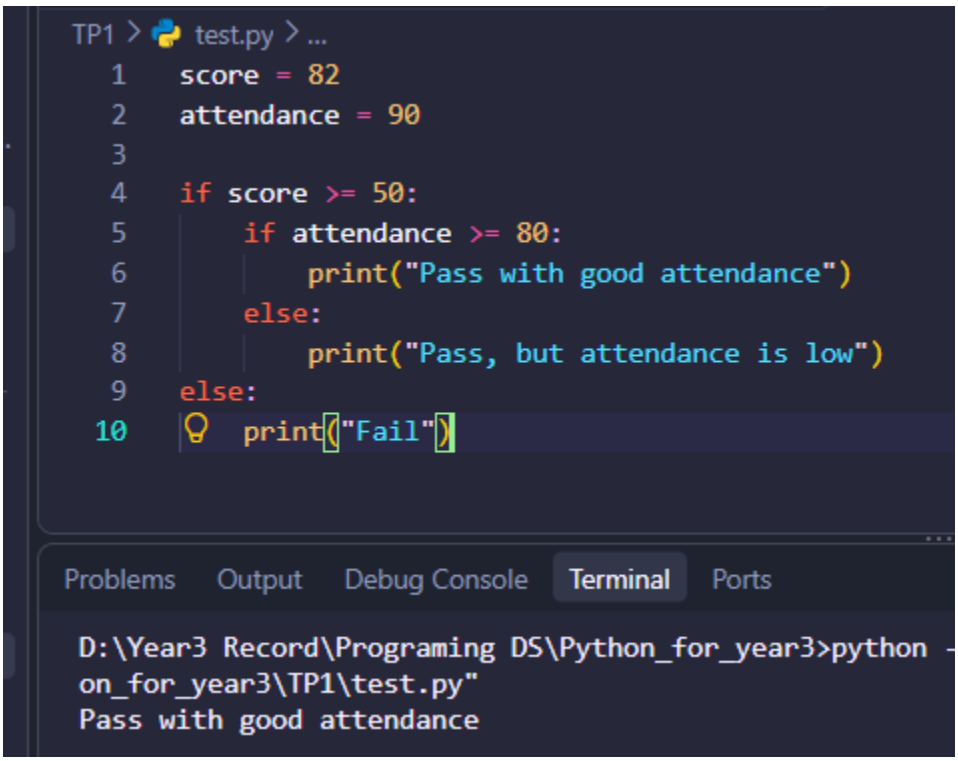
- if runs a block only when a condition is true.
- if-else chooses between two paths, and exactly one runs.
- if-elif-else chooses among many paths, and only the **first** matching block runs.

- Indentation defines the blocks, and each condition line ends with :.
- Order your elif conditions carefully, and use and, or, not, and in to build clear conditions.
- For two simple outcomes, a conditional expression (x if c else y) can replace a short if-else.

Group 8: Python Decision-Making — Part 2
d. Nested if Statements

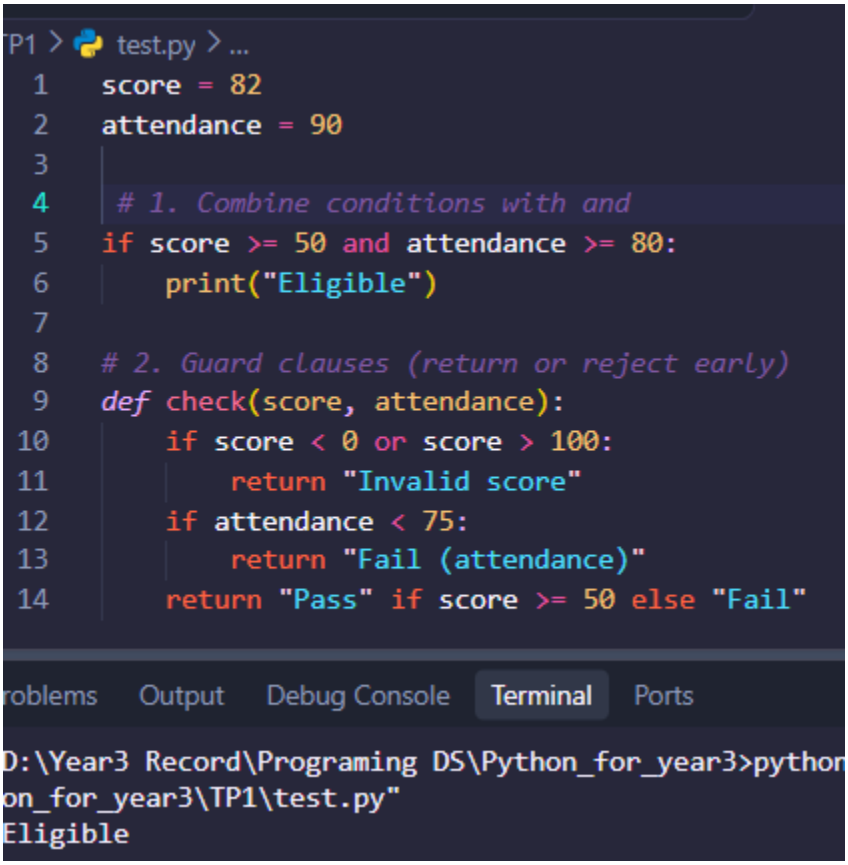
- A nested if is an if placed inside another if, elif, or else block. The inner condition is only checked when the outer one allows it.

Example:



Reducing nesting. Deep nesting is hard to read. Two common fixes:

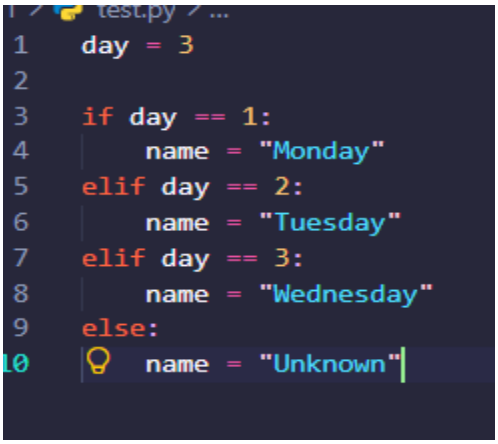
Example:



A good rule: if you go deeper than about 3 levels, restructure the code.

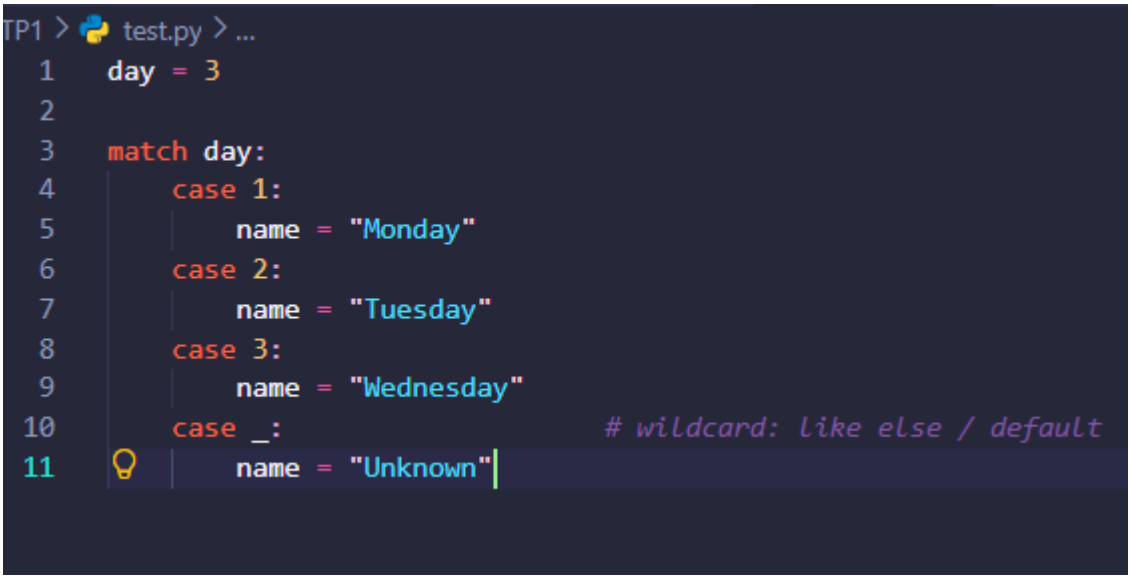
e. Multiple-Branch Selection Using match-case (Python 3.10+) or if-elif-else

- if-elif-else (works in every Python version)



➢ **`match-case` (Python 3.10+)**

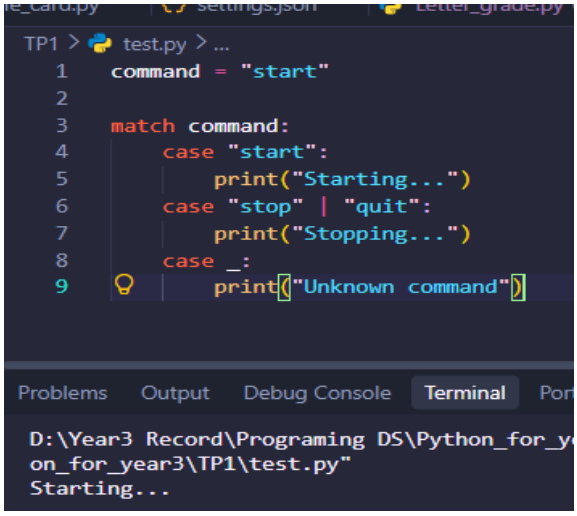
`match` compares one value against several **patterns** from top to bottom and runs the first one that matches.



➢ **Differences from Java's switch:**

- No break needed. Only the first matching case runs, with no fall-through.
- case _ is the default.
- It is more powerful than switch because it matches **structures**, not just values.

Pattern types:



➢ Which to choose?

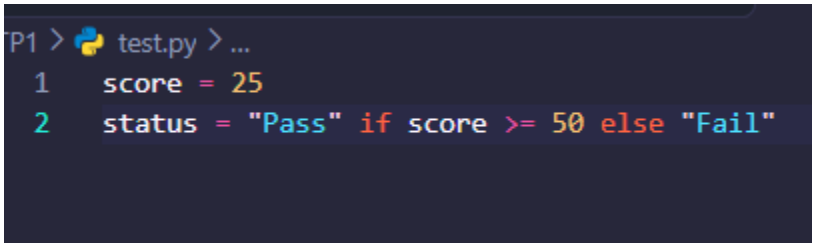
Situation	*Use*
Ranges (score >= 85)	if-elif-else
Many fixed values (menu choices, commands)	match-case
Python older than 3.10	if-elif-else
Matching shapes of data (lists, dicts, objects)	match-case

f. Conditional Expressions

Java and C use the ternary operator:

```
String status = (score >= 50) ? "Pass" :
"Fail";
```

Python puts the condition in the middle:



The pattern is value_if_true if condition else value_if_false.

```python
age = 17
score = 25
input_name = input("Enter a name: ")
print("Adult" if age >= 18 else "Minor")

# Inside an f-string
print(f"Result: {'Pass' if score >= 50 else 'Fail'}")

# Chained
grade = "A" if score >= 85 else "B" if score >= 70 else "C" if score >= 50 else "F"

# Choosing a default value
name = input_name if input_name else "Unknown"
name = input_name or "Unknown"        # shorter, for the same purpose
```

Java	*Python*
c ? a : b	a if c else b
switch (x) { case 1: ... }	match x: case 1: ...
else if	elif

Use it only for simple value choices. Do not use it to run statements. For anything complex, use a normal if-else.

Practical example: a menu-driven student management system.

test.py > ...

```python
students = [
    {"id": 1, "name": "Sophea", "score": 88},
    {"id": 2, "name": "Dara",   "score": 72},
    {"id": 3, "name": "Vanna",  "score": 45},
]


def get_grade(score):
    # conditional expression (chained)
    return "A" if score >= 85 else "B" if score >= 70 else "C" if score >= 50 else "F"

```

```python

def find_student(student_id):
    for s in students:
        if s["id"] == student_id:
            return s
    return None


def show_menu():
    print("\n===== Student Management =====")
    print("1. List students")
    print("2. Add student")
    print("3. Search student by ID")
    print("4. Update score")
    print("5. Exit")


def list_students():
    if not students:                                    # simple if
        print("No students yet.")
        return
    print(f"{'ID':<4}{'Name':<10}{'Score':<7}Grade")
    for s in students:
        status = "Pass" if s["score"] >= 50 else "Fail"   # conditional expression
        print(f"{s['id']:<4}{s['name']:<10}{s['score']:<7}"
              f"{get_grade(s['score'])} ({status})")


def add_student():
    name = input("Name: ").strip()
    if name:                                            # outer if
        score_text = input("Score (0-100): ").strip()
        if score_text.isdigit():                        # nested if
```

```
PT > test.py > ...
def add_student():
    name = input("Name: ").strip()
    if name:                                        # outer if
        score_text = input("Score (0-100): ").strip()
        if score_text.isdigit():                    # nested if
            score = int(score_text)
            if 0 <= score <= 100:                   # nested deeper
                new_id = max((s["id"] for s in students), default=0) + 1
                students.append({"id": new_id, "name": name, "score": score})
                print(f"Added {name} with ID {new_id}.")
            else:
                print("Score must be between 0 and 100.")
        else:
            print("Score must be a whole number.")
    else:
        print("Name cannot be empty.")


def search_student():
    text = input("Enter ID: ").strip()
    if not text.isdigit():
        print("ID must be a number.")
        return
    s = find_student(int(text))
    if s is None:
        print("Student not found.")
    else:
        print(f"{s['name']}: score {s['score']}, grade {get_grade(s['score'])}")


def update_score():
    text = input("Enter ID: ").strip()
    if not text.isdigit():
        print("ID must be a number.")
        return
    s = find_student(int(text))
    if s is None:
        print("Student not found.")
        return
    new_text = input(f"New score for {s['name']} (current {s['score']}): ").strip()
    if new_text.isdigit() and 0 <= int(new_text) <= 100:
        s["score"] = int(new_text)
        print("Score updated.")
    else:
```

```
        print("Student not found.")
        return
    new_text = input(f"New score for {s['name']} (current {s['score']}): ").strip()
    if new_text.isdigit() and 0 <= int(new_text) <= 100:
        s["score"] = int(new_text)
        print("Score updated.")
    else:
        print("Invalid score.")


# ---------- Main program ----------
while True:
    show_menu()
    choice = input("Choose an option: ").strip()

    match choice:                                   # multi-branch selection
        case "1":
            list_students()
        case "2":
            add_student()
        case "3":
            search_student()
        case "4":
            update_score()
        case "5" | "q" | "Q":                       # OR pattern
            print("Goodbye!")
            break
        case _:                                     # default
            print("Invalid choice. Please enter 1-5.")
```

Output:

```
Choose an option: 1
ID  Name      Score  Grade
1   Sophea    88     A (Pass)
2   Dara      72     B (Pass)
3   Vanna     45     F (Fail)

===== Student Management =====
1. List students
2. Add student
3. Search student by ID
4. Update score
5. Exit
Choose an option: 2
Name: Piseth
Score (0-100): 90
Added Piseth with ID 4.
```

```
===== Student Management =====
1. List students
2. Add student
3. Search student by ID
4. Update score
5. Exit
Choose an option: 3
Enter ID: 4
Piseth: score 90, grade A

===== Student Management =====
1. List students
2. Add student
3. Search student by ID
4. Update score
5. Exit
Choose an option: 4
Enter ID: 4
New score for Piseth (current 90): 100
Score updated.

===== Student Management =====
1. List students
2. Add student
3. Search student by ID
4. Update score
5. Exit
Choose an option: 1
ID  Name      Score  Grade
1   Sophea    88     A (Pass)
2   Dara      72     B (Pass)
3   Vanna     45     F (Fail)
4   Piseth    100    A (Pass)
```

Key Takeaways

- **Nested if** checks a second condition only when the first is satisfied. Keep nesting shallow by using and or early returns.
- **match-case** (Python 3.10+) selects among many branches with no break or fall-through, and case _ is the default. It also supports |, guards (if), and structure patterns.
- Use if-elif-else for **ranges** and for Python versions before 3.10, and match-case for **fixed values** like menu options.
- A bare name in a case captures a value instead of comparing to it.

- **Conditional expression:** Java's c ? a : b becomes a

Group 9: Python Loops and Loop Control

- A loop repeats a block of code. Python has two loops, while and for. As with if, the block is defined by a colon and indentation.

a. The while Loop

- Repeats **as long as** the condition is true. Use it when you do not know in advance how many repetitions you need.

Syntax:

while condition:

runs again and again while condition is True

Example:



Key points:

- The condition is checked **before** each repetition, so the body may run zero times.
- Something inside the loop must eventually make the condition false, otherwise you get an infinite loop.
- Any value works as a condition (while items: runs while a list is not empty).

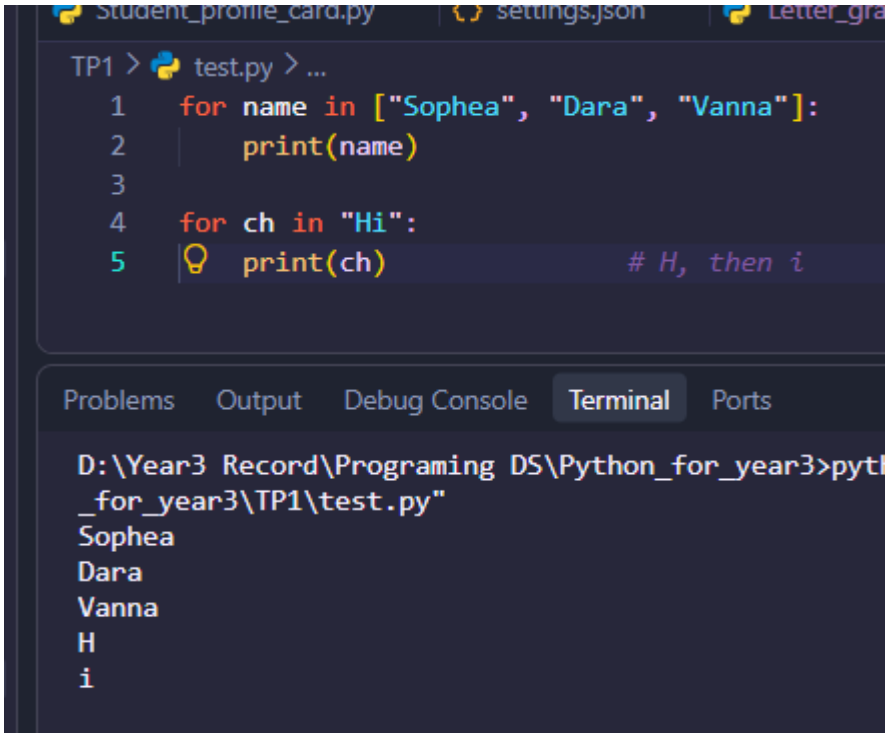
b. The for Loop

➢ Goes through the items of any **iterable** (list, tuple, string, dict, set, range, and so on). Use it when you have a collection or a known number of repetitions.

Syntax:

```
for item in iterable:
        # runs once per item
```

Example:

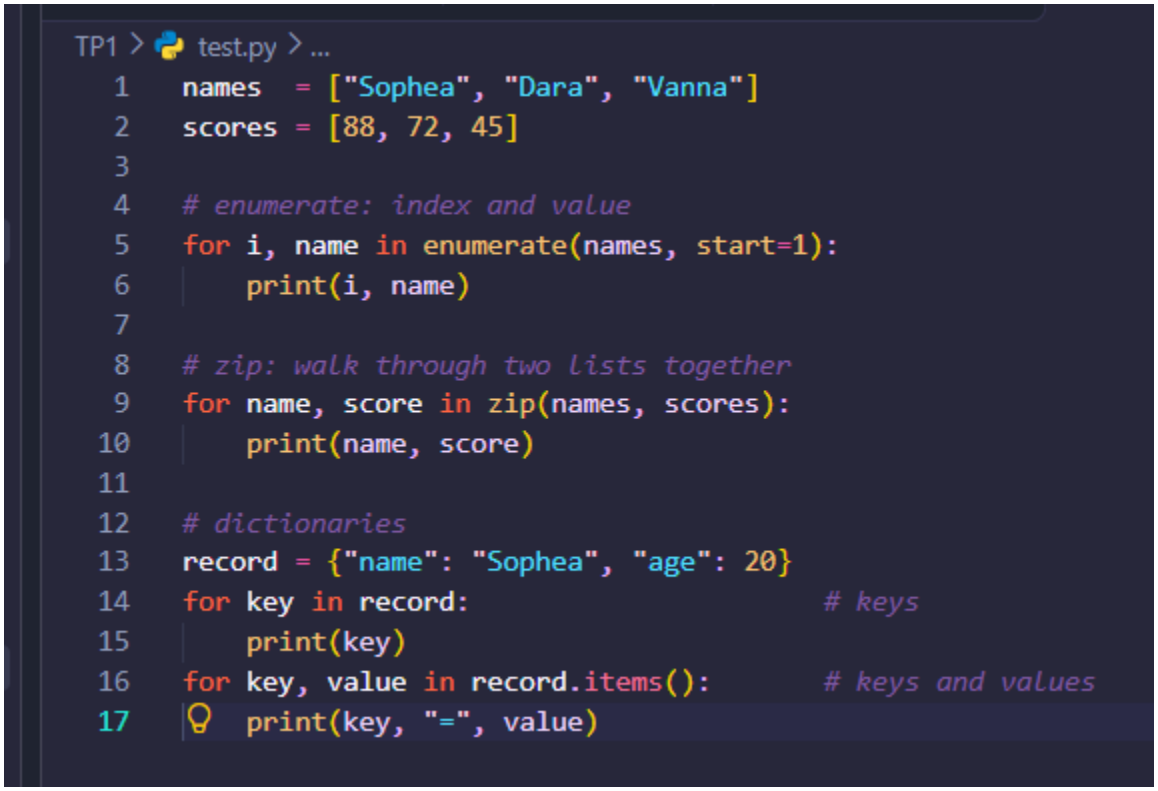


range() generates numbers:

Call	Produces
range(5)	0,1,2,3,4
range(1, 6)	1,2,3,4,5
range(0, 10, 2)	0,2,4,6,8
range(5, 0, -1)	5,4,3,2,1

The end value is never included.

Useful helpers:



Output:

```
D:\Year3 Record\Programing DS\Python_for_year3>python -u "d:\Year3
_for_year3\TP1\test.py"
1 Sophea
2 Dara
3 Vanna
Sophea 88
Dara 72
Vanna 45
name
age
name = Sophea
age = 20
```

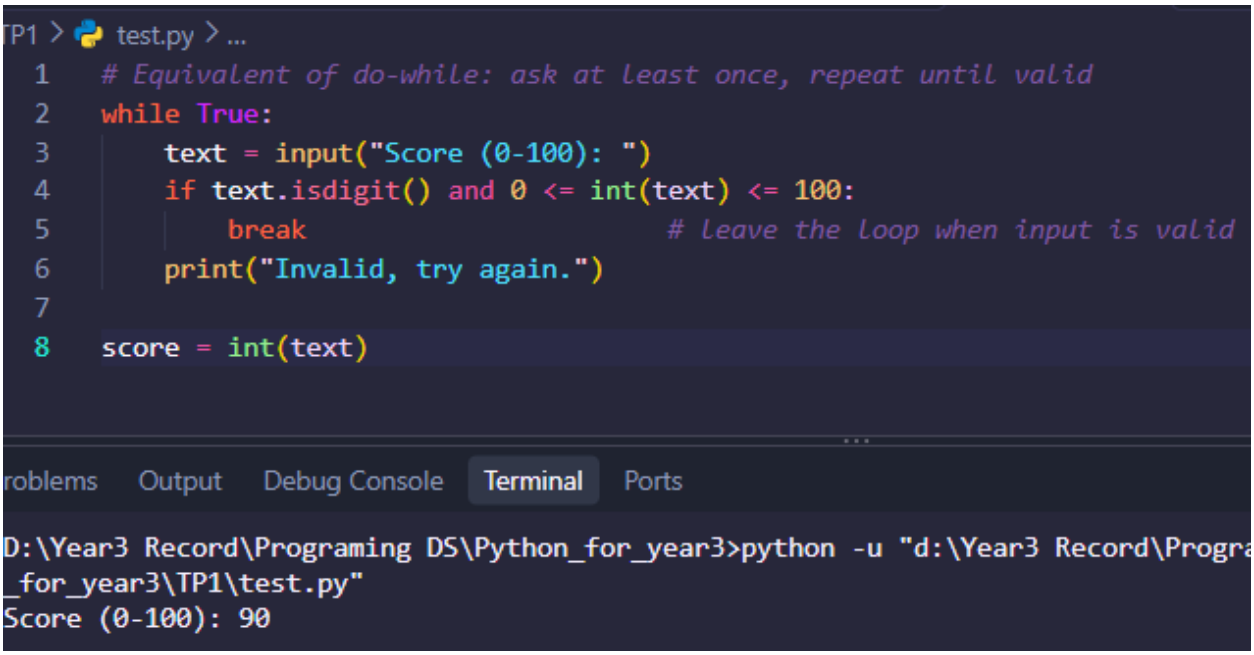
Cautions:

- Do not add or remove items from a list while looping over it. Loop over a copy (for x in items[:]:) or build a new list.
- The loop variable still exists after the loop ends.

c. Repetition with for and while (Python has no native do-while loop)

Use for when...	Use while when...
You have a collection to go through	You repeat until a condition changes
You know the number of repetitions	The number of repetitions is unknown
Example: print every student	Example: keep asking until the input is valid

- Java's do { ... } while (cond); runs the body **at least once**. Python has no such loop, so the standard replacement is while True with a break:

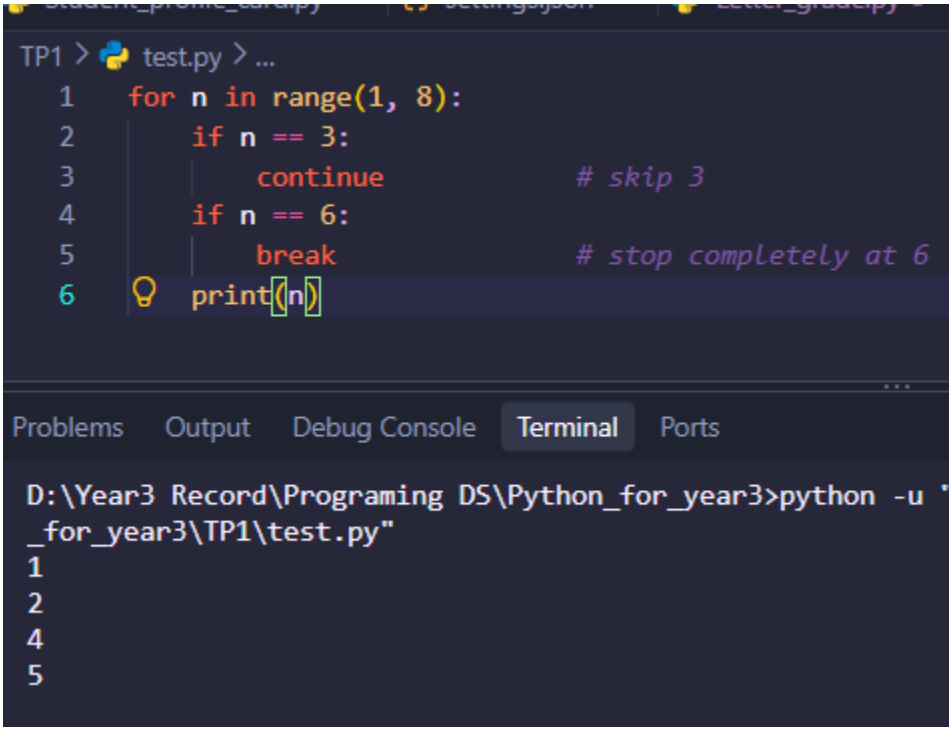


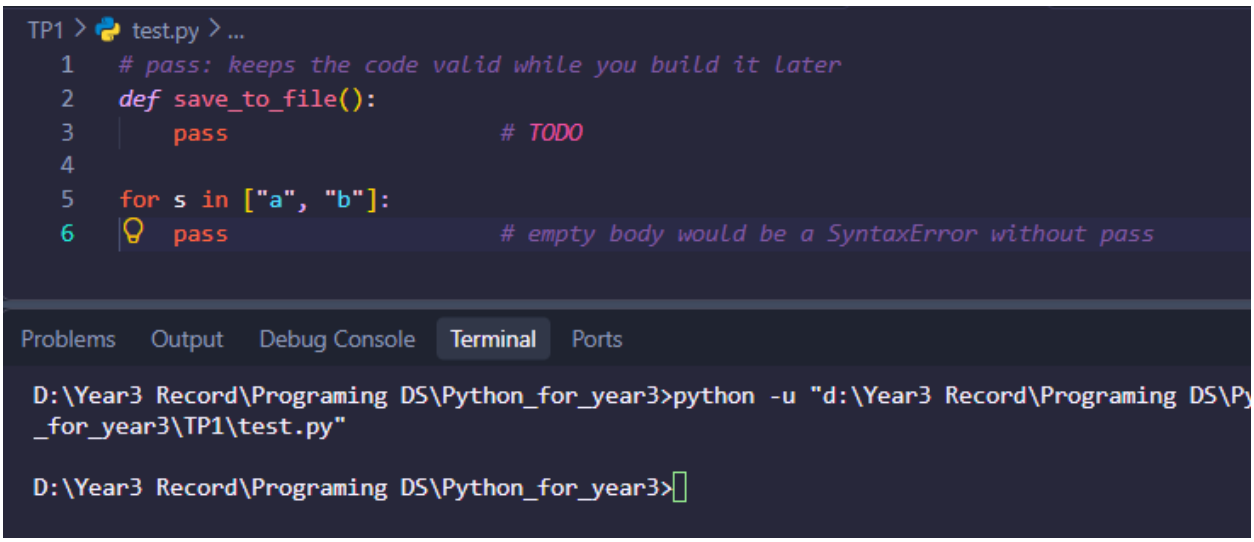
- Another way is to run the body once before a normal while, but that duplicates code, so while True + break is preferred.

d. Loop Control Statements: break, continue, and pass

Statement	Effect
break	Leaves the loop immediately
continue	Skips the rest of this repetition and goes to the next one
pass	Does nothing; a placeholder where a statement is required

Example:

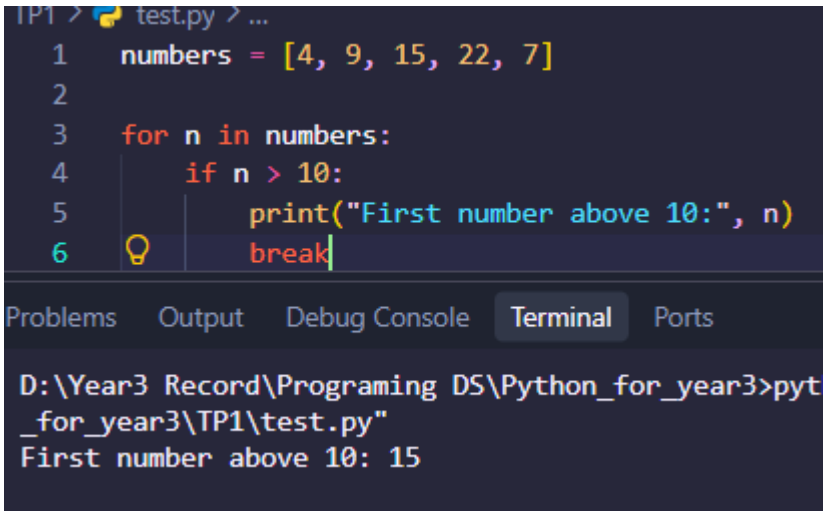




e. The break Statement

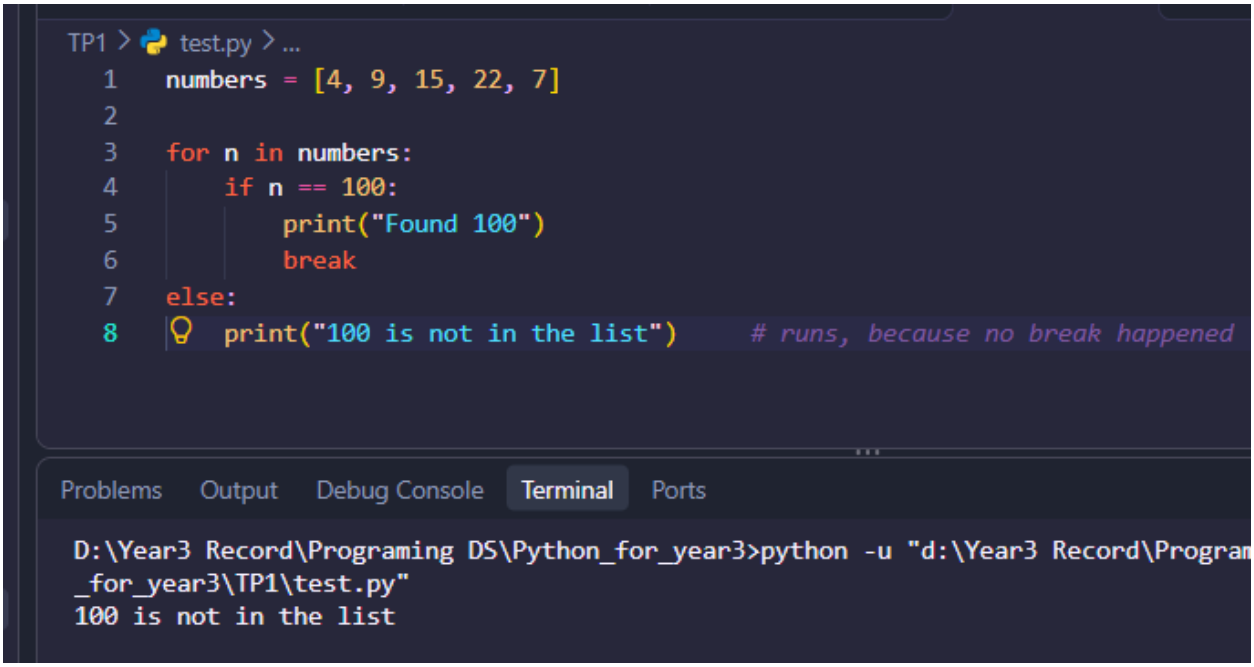
- break ends the **innermost** loop right away, and execution continues after the loop. It is mostly used for searching and for stopping when a goal is reached.

Example:



- **Loop else** (a Python feature): the else block of a loop runs only if the loop finished **without** hitting break. It is perfect for "not found" cases.

Example:



Note: break and continue affect only the innermost loop. Python has no labeled break like Java.

Practical example: iterating through student objects, calculating averages, and searching for a student.

test.py > ...

```python
class Student:
    def __init__(self, name, scores):
        self.name = name
        self.scores = scores

    def average(self):
        return sum(self.scores) / len(self.scores)


students = [
    Student("Sophea", [88, 92, 85]),
    Student("Dara",   [72, 65, 80]),
    Student("Vanna",  [45, 60, 55]),
    Student("Chenda", []),                  # no scores yet
]

# ---- 1. for loop: iterate through objects and calculate averages ----
print("--- Averages ---")
total = 0
counted = 0
for i, s in enumerate(students, start=1):
    if not s.scores:                        # continue: skip students without scores
        print(f"{i}. {s.name}: no scores yet")
        continue
    avg = s.average()
    total += avg
    counted += 1
    print(f"{i}. {s.name}: {avg:.2f}")

print(f"Class average: {total / counted:.2f}")


# ---- 2. for-else with break: search for a student ----
print("\n--- Search (for) ---")
target = "Vanna"
for s in students:
    if s.name == target:
        print(f"Found {s.name}, average {s.average():.2f}")
        break                               # stop searching once found
else:
    print(f"{target} not found")
```

```python
# ---- 3. while loop: the same search using an index ----
print("\n--- Search (while) ---")
target = "Kosal"
i = 0
while i < len(students) and students[i].name != target:
    i += 1

if i < len(students):
    print("Found at position", i)
else:
    print(f"{target} not found")


# ---- 4. while True + break: do-while style input ----
print("\n--- Add a score ---")
while True:
    text = input("New score for Dara (0-100): ")
    if text.isdigit() and 0 <= int(text) <= 100:
        students[1].scores.append(int(text))
        break
    print("Invalid, try again.")
print("Dara's scores:", students[1].scores)
```

Output:

```
D:\Year3 Record\Programing DS\Python_for_year3>pyth
_for_year3\TP1\test.py"
--- Averages ---
1. Sophea: 88.33
2. Dara: 72.33
3. Vanna: 53.33
4. Chenda: no scores yet
Class average: 71.33

--- Search (for) ---
Found Vanna, average 53.33
```

```
--- Search (while) ---
Kosal not found

--- Add a score ---
New score for Dara (0-100): abc
Invalid, try again.
New score for Dara (0-100): 90
Dara's scores: [72, 65, 80, 90]

D:\Year3 Record\Programing DS\Python_for_year3>
```

- **Important Differences Between Java and Python**

Topic	Java	Python
Blocks	{ } braces	Colon and indentation
Condition	Needs parentheses: while (x < 5)	No parentheses needed: while x < 5:
Counting loop	for (int i = 0; i < 5; i++)	for i in range(5):
For-each	for (String s : list)	for s in list:
Increment	i++, ++i	Not available; use i += 1
Condition type	Must be boolean	Any value (truthy or falsy)
do-while	Built in	Not available; use while True + break
Loop else	Not available	Runs if the loop ended without break
Labeled break/continue	Supported	Not supported
Empty block	{ }	Needs pass
Loop variable after loop	Out of scope if declared in the for	Still accessible

Iterating with an index	Usual style	Use enumerate() instead of range(len(...))
Booleans	true, false	True, False (capitalized)

Example:

```java
// Java
int sum = 0;
for (int i = 1; i <= 5; i++) {
    sum += i;
}
```

```python
test.py > ...
# Python
total = 0
for i in range(1, 6):       # 6, because the end is excluded
    total += i
```

Common Mistakes

Mistake	Problem	Fix
Forgetting to update the variable in while	Infinite loop	Change the loop variable inside the body
range(1, 5) expecting 5	Stops at 4	Use range(1, 6)
Missing : after for/while	SyntaxError	Add the colon
Modifying a list while looping over it	Items get skipped	Loop over a copy or build a new list
Using i++	SyntaxError	Use i += 1
Dividing by len(...) of an empty list	ZeroDivisionError	Check if not scores: first

Key Takeaways

- Use **for** to go through a collection or a known range, and **while** to repeat until a condition changes.
- **do-while** does not exist, so use while True with break.
- **break** leaves the loop, **continue** skips to the next repetition, and **pass** does nothing.
- A loop **else** runs only when no break happened, which is ideal for “not found” messages.
- Prefer enumerate() and zip() over manual index handling.
- Compared with Java, Python uses indentation, range(), no ++, and any truthy value as a condition.

Conclusion

These nine topics form the foundation of Python programming. Data types and variables store information, operators combine and compare it, decision statements choose a path, and loops repeat work. Object-oriented features such as classes, properties, and abstract methods organize all of this into reusable, well-protected designs.

The practical examples built up from simple values to a complete menu-driven student management system, showing how each concept fits into a real program. With these fundamentals in place, natural next steps are functions and modules, exception handling, file input and output, and comprehension syntax.

Thank you for your attention. Questions are welcome.